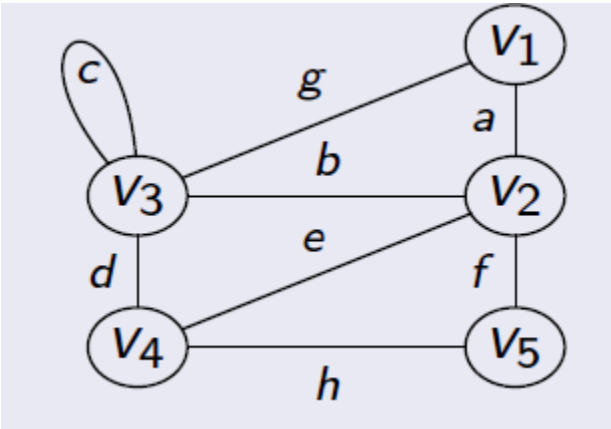
Unidade II
Caminhos e Circuitos

Prof. Henrique Batista da Silva

Implementações adaptadas de [Ziviani, 2006]

Caminhos e circuitos

- Sequência de arestas: sequência alternada de vértices e arestas começando e terminando com vértice. Cada aresta é incidente ao vértice que a precede e a antecede:
  - Ex.: $v_1\ a\ v_2\ a\ v_1\ g\ v_3$
- Caminho: sequência de arestas no qual nenhuma aresta aparece mais de uma vez:
  - Ex.:$v_1\ a\ v_2\ b\ v_3\ c\ v_3\ d\ v_4\ e\ v_2\ f\ v_5$
  - Caminho aberto: vértice inicial é diferente do vértice final
    - Ex.: $v_1\ a\ v_2\ b\ v_3\ c\ v_3$
  - Caminho fechado: caminhos que começam e terminam no mesmo vértice:
    - Ex.: $v_1\ a\ v_2\ b\ v_3\ c\ v_3\ g\ v_1$



Pesquisa em grafos

- Dado um grafo $G = (V, E)$, direcionado ou não direcionado:
  - Meta: metodicamente, explorar todos os vértices e todas as arestas.

Busca em Profundidade

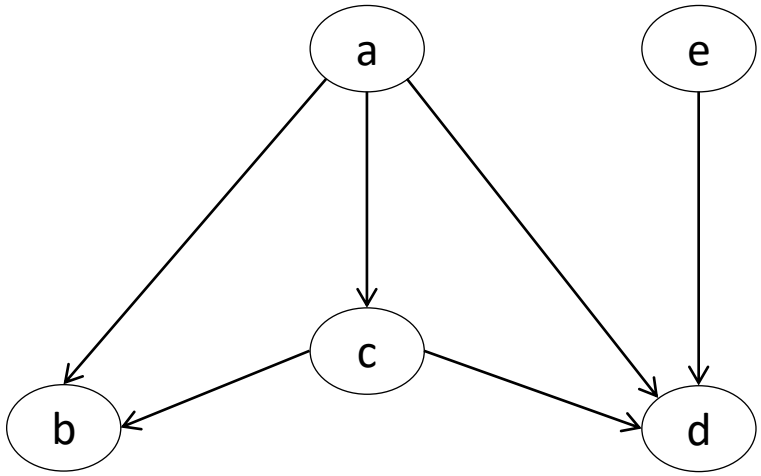
- As arestas são exploradas a partir do vértice v mais recentemente descoberto que ainda possui arestas não exploradas saindo dele.
- Quando todas as arestas adjacentes a v tiverem sido exploradas a busca anda para trás para explorar vértices que saem do vértice do qual v foi descoberto

Busca em Profundidade

- Para acompanhar o progresso do algoritmo cada vértice é colorido de branco, cinza ou preto.
- Todos os vértices são inicializados como branco.
- Quando um vértice é *descoberto* pela primeira vez ele torna-se cinza, e é tornado preto quando sua lista de adjacentes tenha sido completamente examinada.

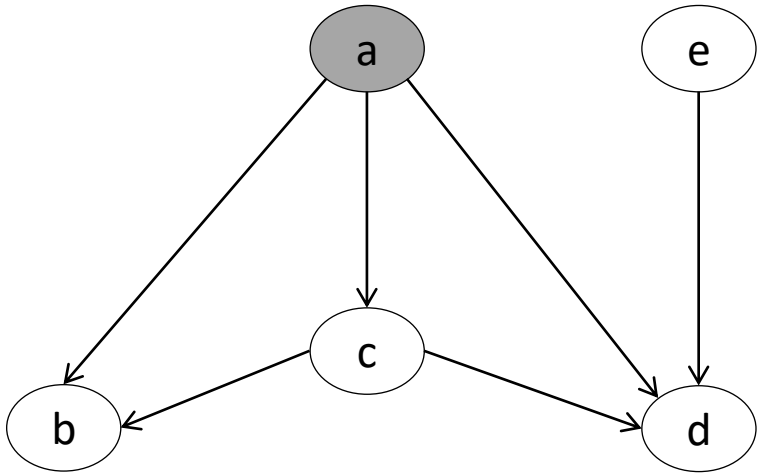
Busca em Profundidade

- Todos os vértice são inicializados com branco.
- Quando um vértice é visitado pela primeira vez ele torna-se cinza.
- Quando sua lista de adjacentes foi totalmente explorada ele torna-se vermelho



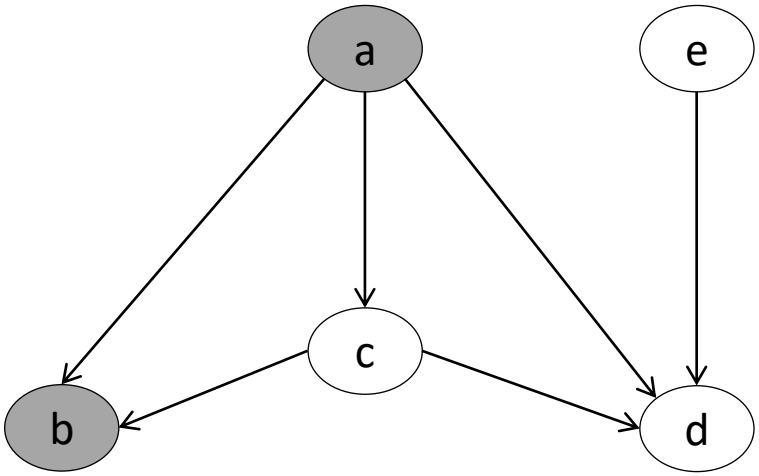
Busca em Profundidade

- Todos os vértice são inicializados com branco.
- Quando um vértice é visitado pela primeira vez ele torna-se cinza.
- Quando sua lista de adjacentes foi totalmente explorada ele torna-se vermelho



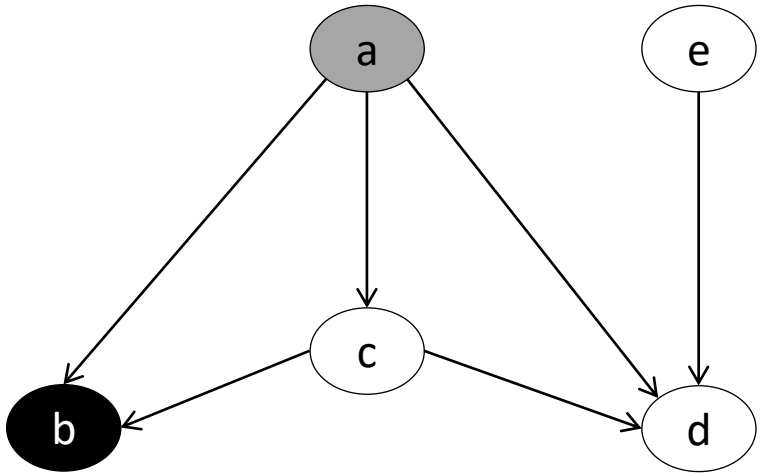
Busca em Profundidade

- Todos os vértice são inicializados com branco.
- Quando um vértice é visitado pela primeira vez ele torna-se cinza.
- Quando sua lista de adjacentes foi totalmente explorada ele torna-se vermelho



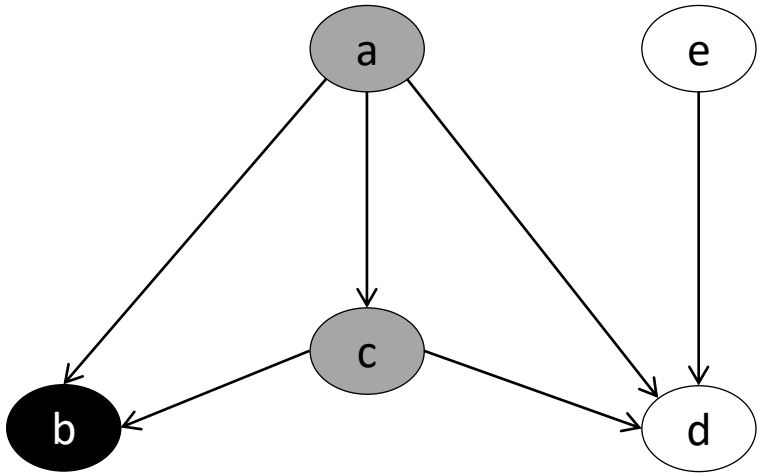
Busca em Profundidade

- Todos os vértice são inicializados com branco.
- Quando um vértice é visitado pela primeira vez ele torna-se cinza.
- Quando sua lista de adjacentes foi totalmente explorada ele torna-se vermelho



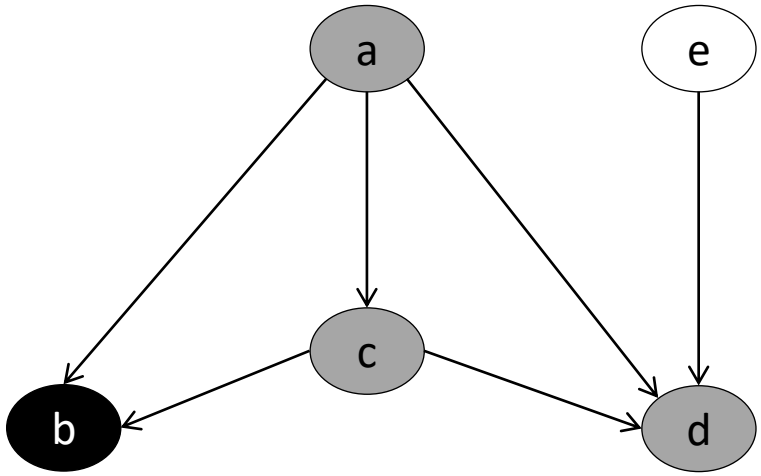
Busca em Profundidade

- Todos os vértice são inicializados com branco.
- Quando um vértice é visitado pela primeira vez ele torna-se cinza.
- Quando sua lista de adjacentes foi totalmente explorada ele torna-se vermelho



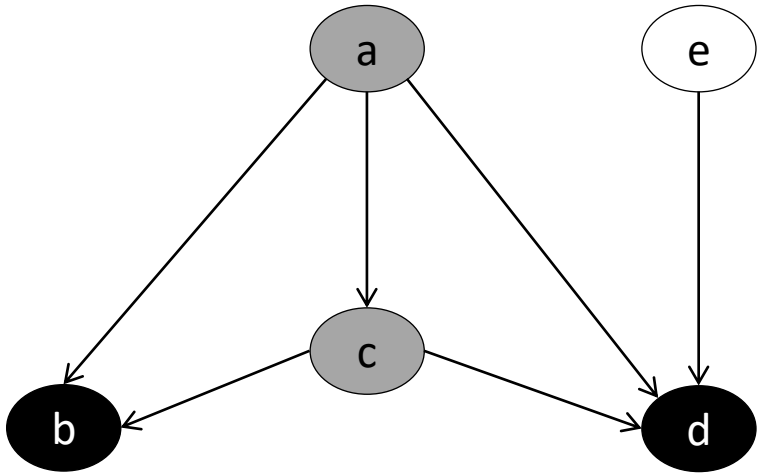
Busca em Profundidade

- Todos os vértice são inicializados com branco.
- Quando um vértice é visitado pela primeira vez ele torna-se cinza.
- Quando sua lista de adjacentes foi totalmente explorada ele torna-se vermelho



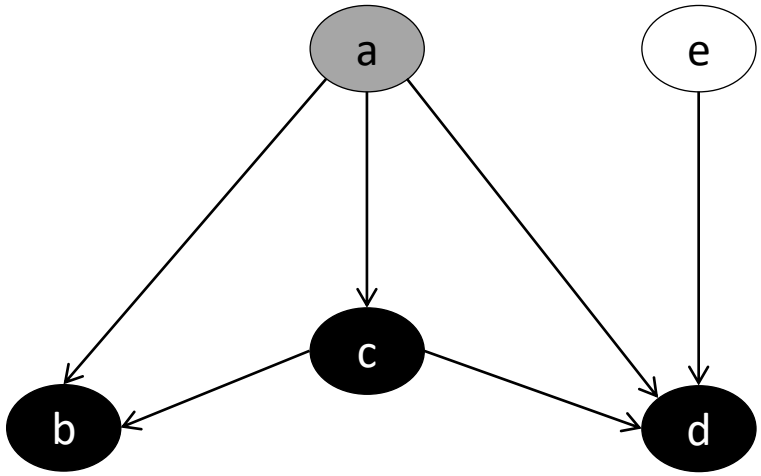
Busca em Profundidade

- Todos os vértice são inicializados com branco.
- Quando um vértice é visitado pela primeira vez ele torna-se cinza.
- Quando sua lista de adjacentes foi totalmente explorada ele torna-se vermelho



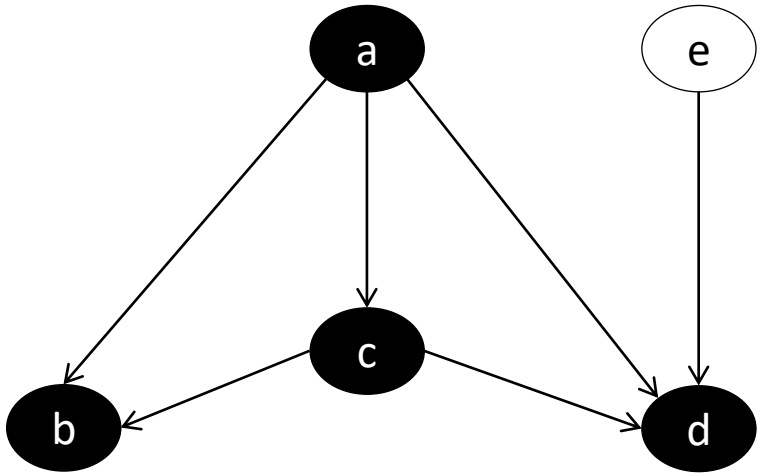
Busca em Profundidade

- Todos os vértice são inicializados com branco.
- Quando um vértice é visitado pela primeira vez ele torna-se cinza.
- Quando sua lista de adjacentes foi totalmente explorada ele torna-se vermelho



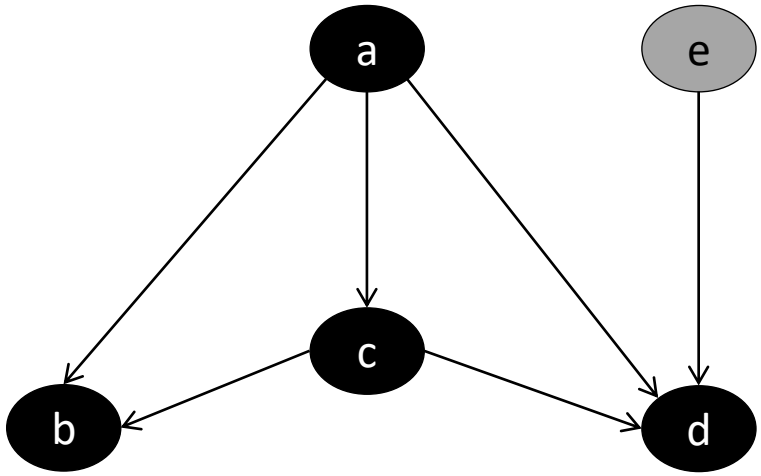
Busca em Profundidade

- Todos os vértice são inicializados com branco.
- Quando um vértice é visitado pela primeira vez ele torna-se cinza.
- Quando sua lista de adjacentes foi totalmente explorada ele torna-se vermelho



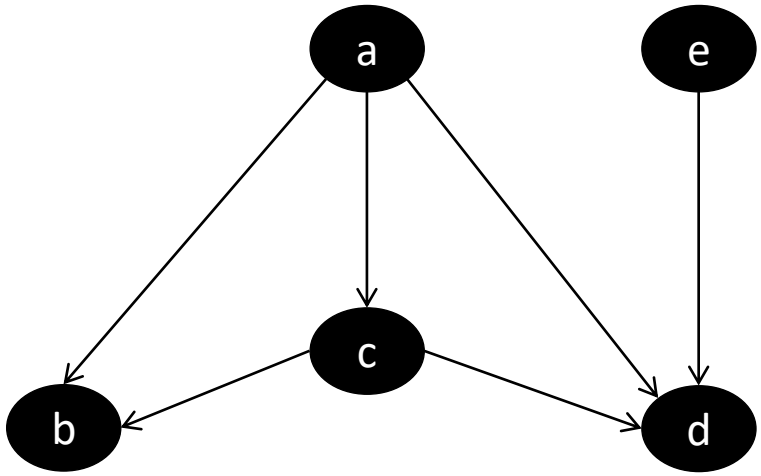
Busca em Profundidade

- Todos os vértice são inicializados com branco.
- Quando um vértice é visitado pela primeira vez ele torna-se cinza.
- Quando sua lista de adjacentes foi totalmente explorada ele torna-se vermelho



Busca em Profundidade

- Todos os vértice são inicializados com branco.
- Quando um vértice é visitado pela primeira vez ele torna-se cinza.
- Quando sua lista de adjacentes foi totalmente explorada ele torna-se vermelho



Busca em Profundidade

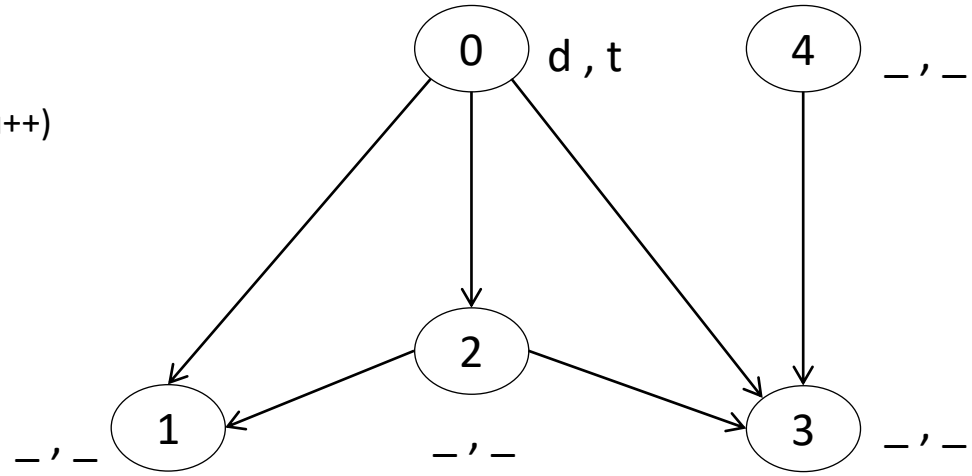
- O tempo de descoberta $d[v]$ é o momento em que o vértice v foi visitado pela primeira vez
- O tempo de término do exame da lista de adjacentes $t[v]$ é o momento em que a visita a toda lista de vértices adjacentes a v foi concluída.
- Estes registros são inteiros entre 1 e $2|V|$ pois existe um evento de descoberta e um evento de término para cada um dos $|V|$ vértices.

Busca em Profundidade

```
public class BuscaEmProfundidade
  {
    public static const byte branco = 0;
    public static const byte cinza  = 1;
    public static const byte preto  = 2;

    private int []d;
    private int []t;
    private int []antecessor;
    private Grafo grafo;

    public BuscaEmProfundidade (Grafo grafo)
    {
      this.grafo = grafo; int n = this.grafo.get_numVertices();
      d = new int[n]; t = new int[n]; antecessor = new int[n];
    }

    public int d (int v) { return this.d[v]; }
    public int t (int v) { return this.t[v]; }
    public int antecessor (int v) { return this.antecessor[v]; }
```

Busca em Profundidade

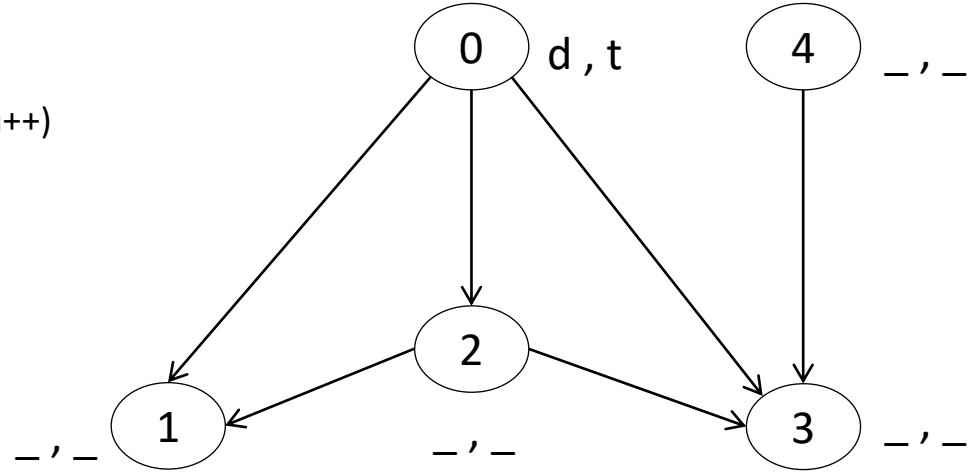
```
public void buscaEmProfundidade ()
    {
        int tempo = 0; int []cor = new int[this.grafo. get_numVertices ()];

        for (int u = 0; u < grafo. get_numVertices (); u++)
        {
            cor[u] = branco; this.antecessor[u] = -1;
        }

        for (int u = 0; u < grafo. get_numVertices (); u++)
            if (cor[u] == branco)
                tempo = this.visitaDfs (u, tempo, cor);
    }
```



Busca em Profundidade

```
public void buscaEmProfundidade ()
    {
        int tempo = 0; int []cor = new int[this.grafo. get_numVertices ()];

        for (int u = 0; u < grafo. get_numVertices (); u++)
        {
            cor[u] = branco; this.antecessor[u] = -1;
        }

        for (int u = 0; u < grafo. get_numVertices (); u++)
            if (cor[u] == branco)
                tempo = this.visitaDfs (u, tempo, cor);
    }
```

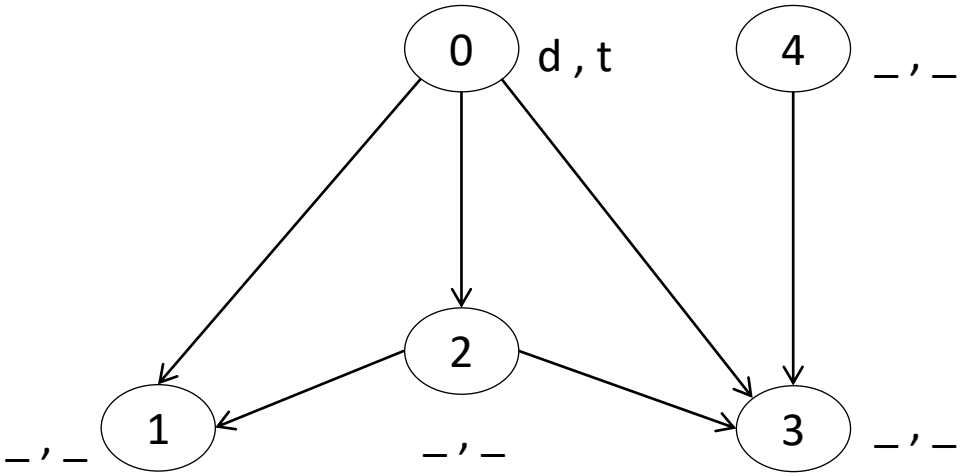
Início da chamada da função visitaDfs



Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

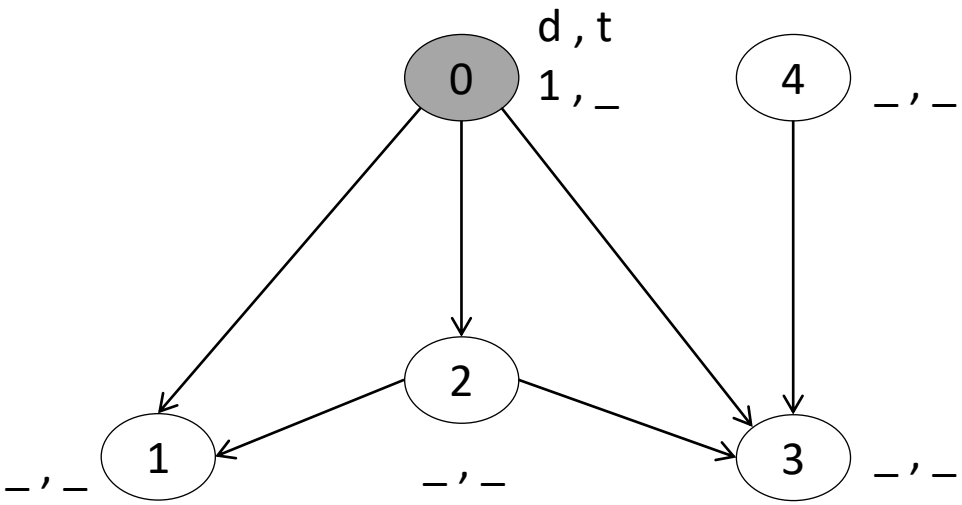
vértice atual → u = 0
tempo = 0
cor [u] = branco



Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 1
cor [u] = cinza

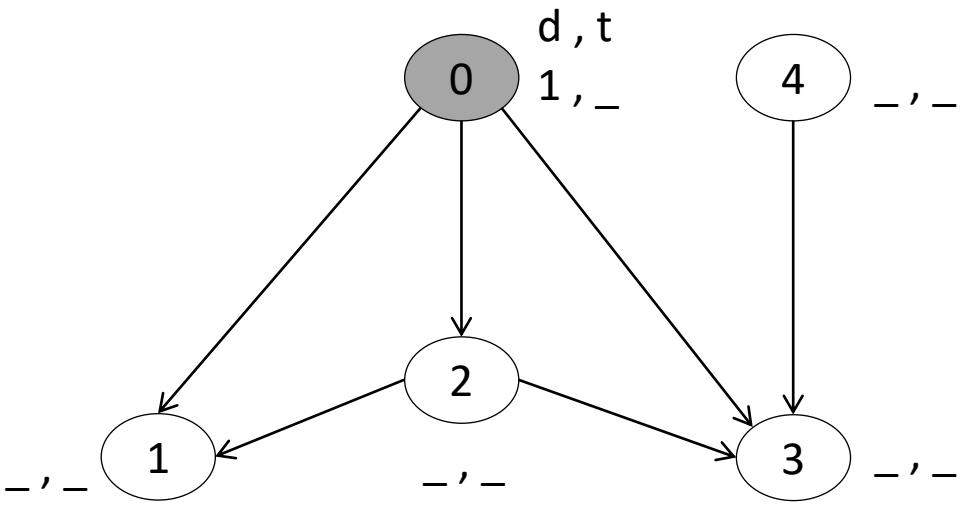


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

Verifica se lista de adj do vértice u está vazia

u = 0
tempo = 1
cor [u] = cinza

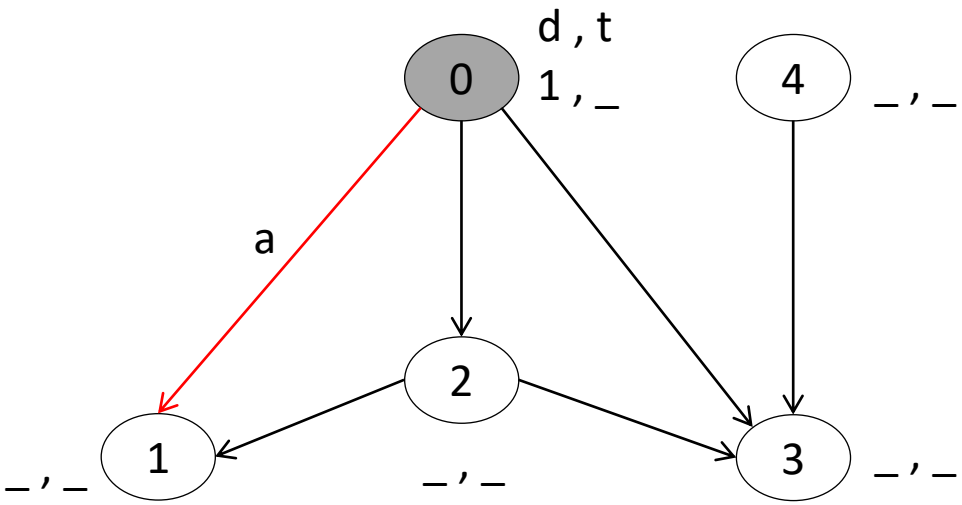


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 1
cor [u] = cinza

a = aresta (0 -> 1)
v = 1

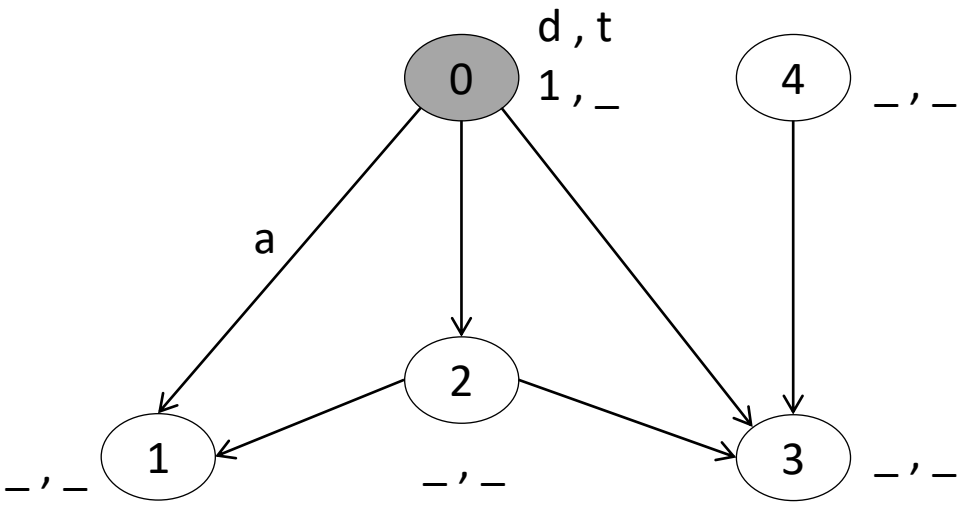


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 1
cor [u] = cinza

a = aresta (0 -> 1)
v = 1

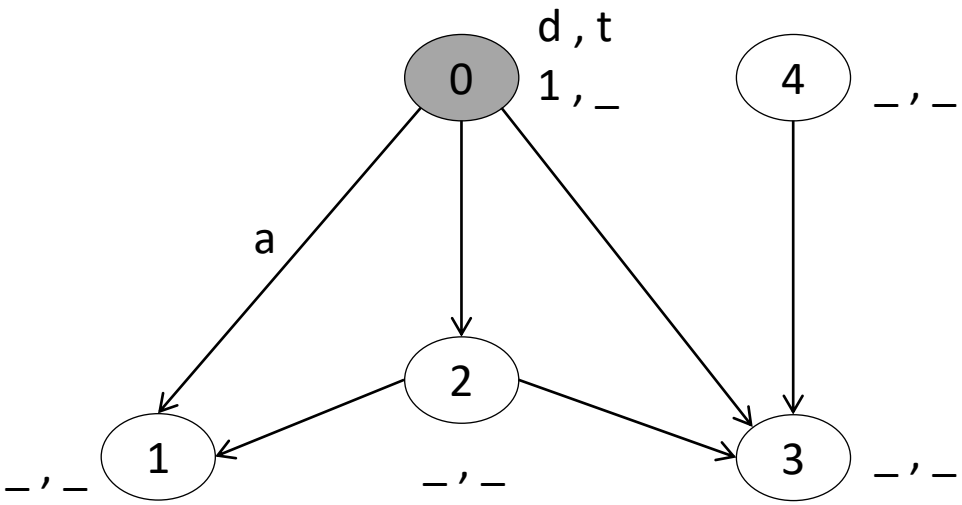


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 1
cor [u] = cinza

a = aresta (0 -> 1)
v = 1

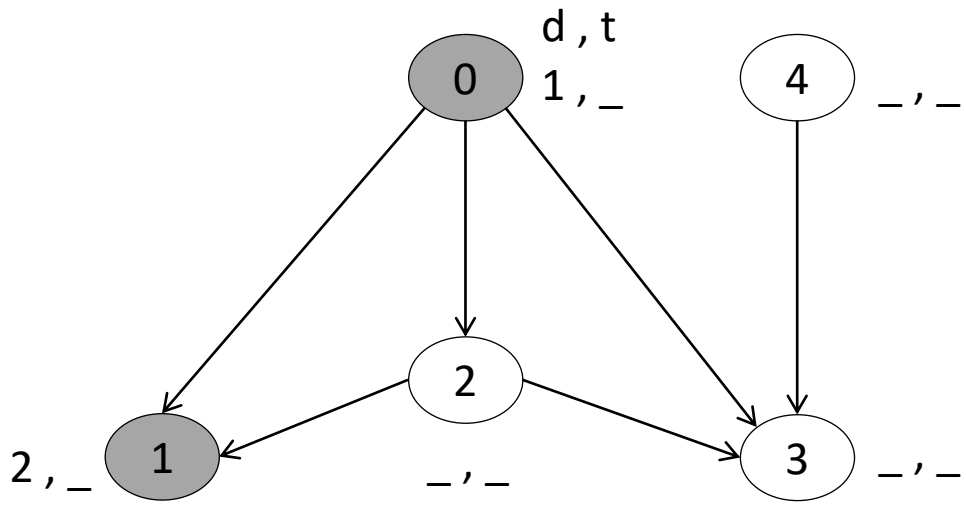


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 1
tempo = 2
cor [u] = cinza

a =
v =

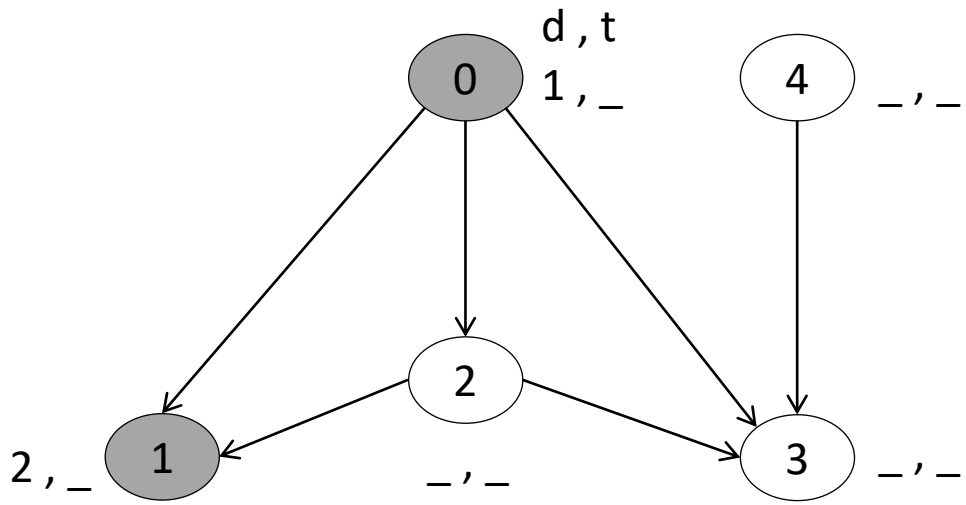


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 1
tempo = 2
cor [u] = cinza

a =
v =

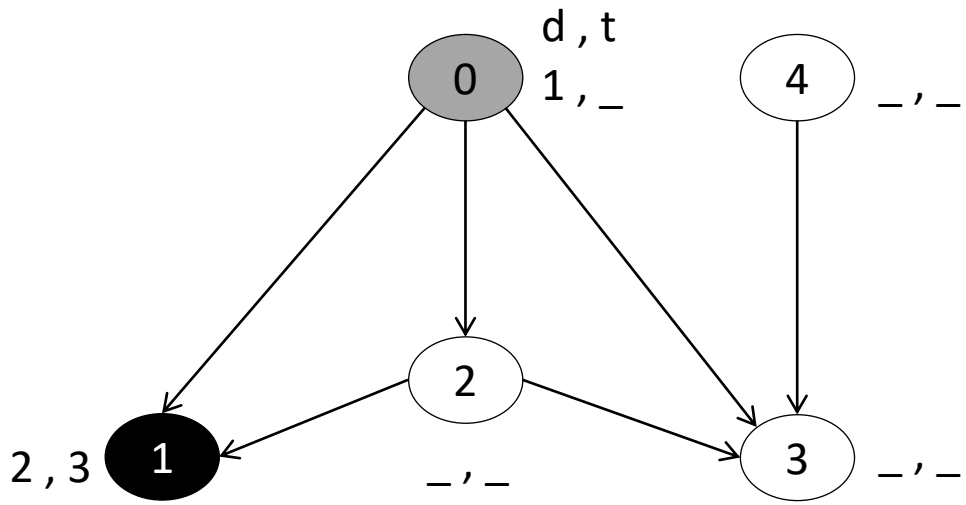


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 1
tempo = 3
cor [u] = preto

a =
v =

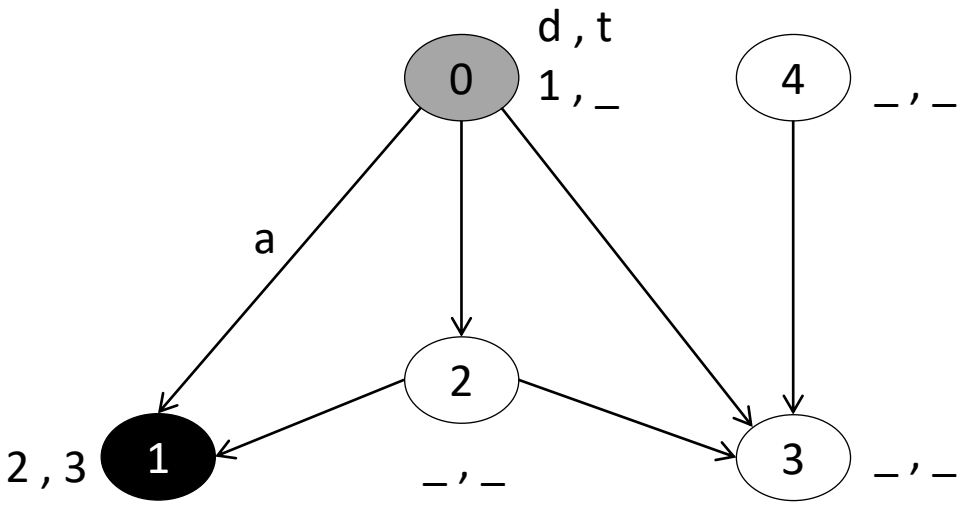


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 3
cor [u] = cinza

a = aresta (0 -> 1)
v = 1

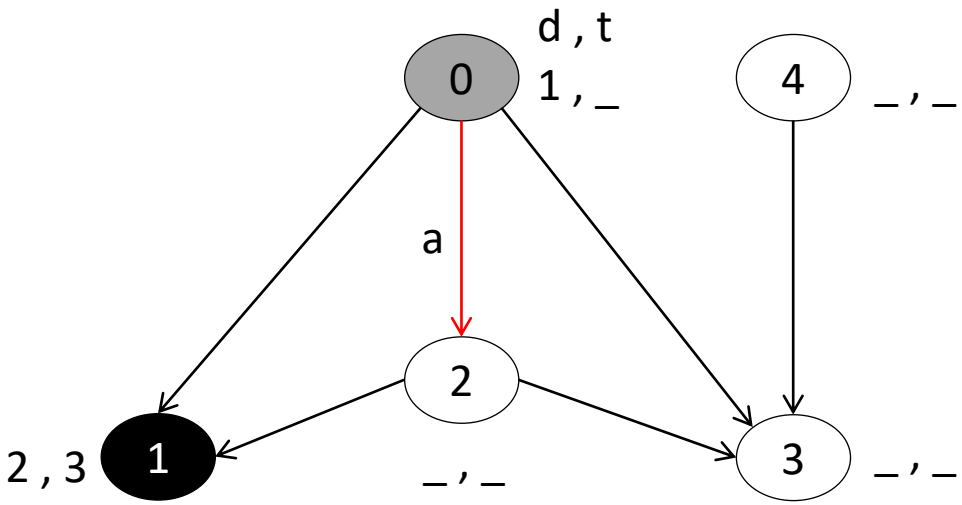


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 3
cor [u] = cinza

a = aresta (0 -> 2)
v = 1

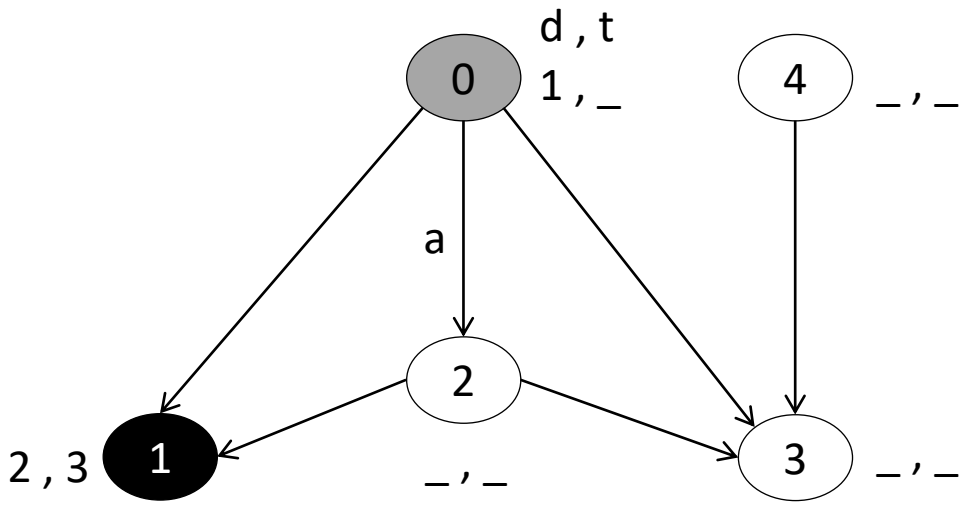


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 3
cor [u] = cinza

a = aresta (0 -> 2)
v = 1

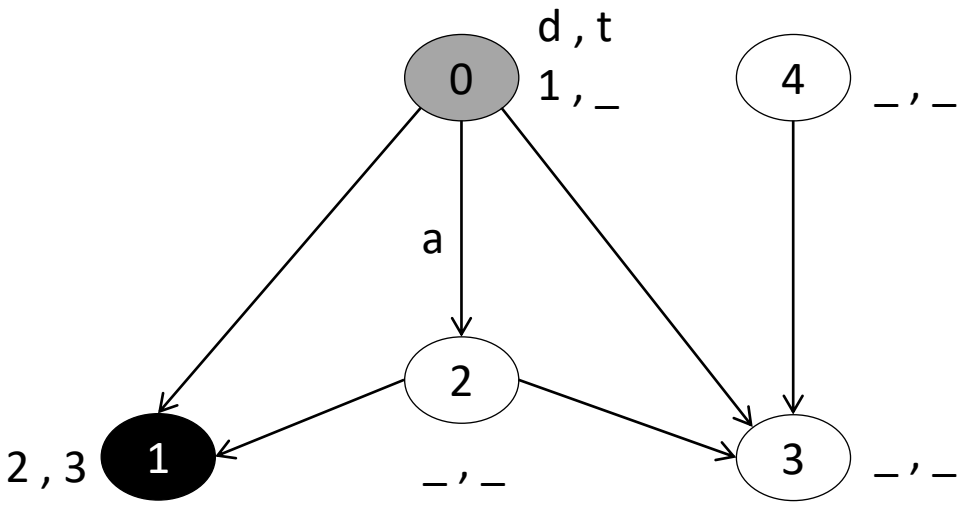


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 3
cor [u] = cinza

a = aresta (0 -> 2)
v = 2

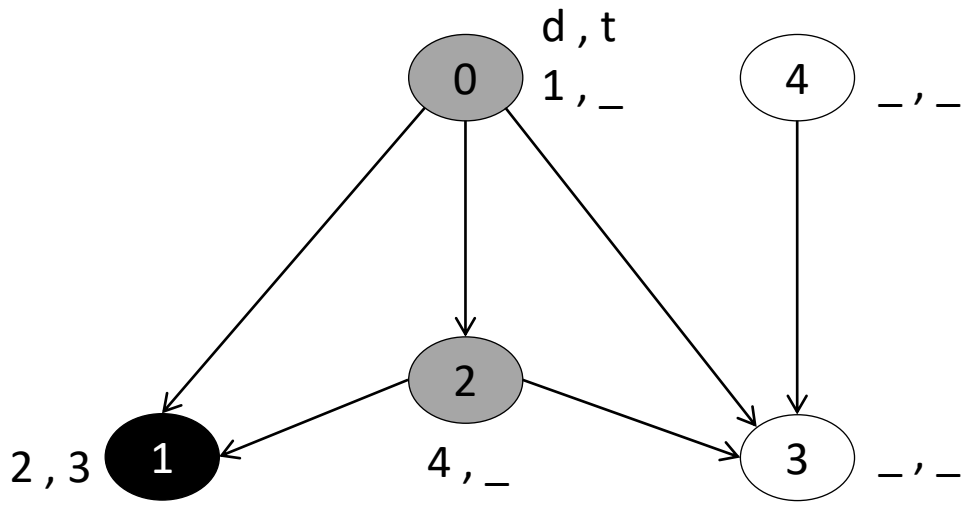


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 2
tempo = 4
cor [u] = cinza

a =
v =

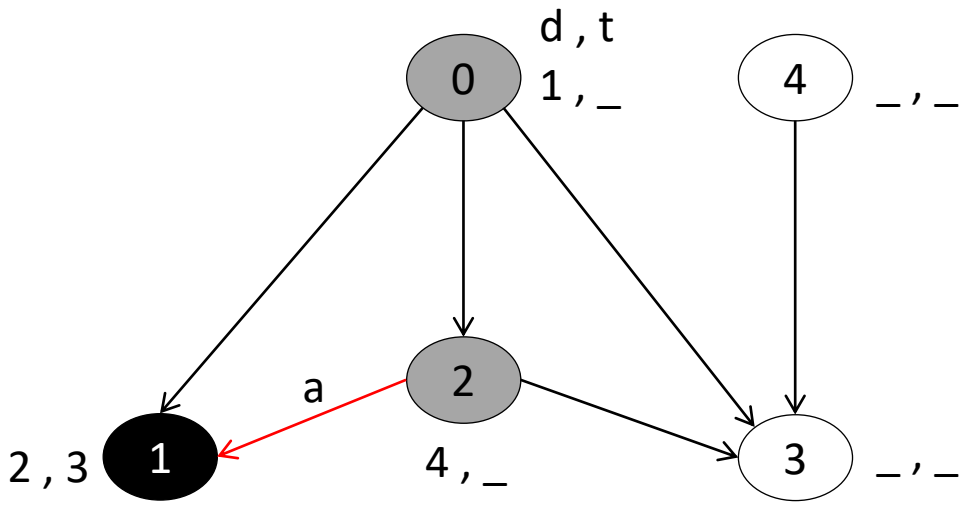


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 2
tempo = 4
cor [u] = cinza

a = aresta (2 -> 1)
v = 1

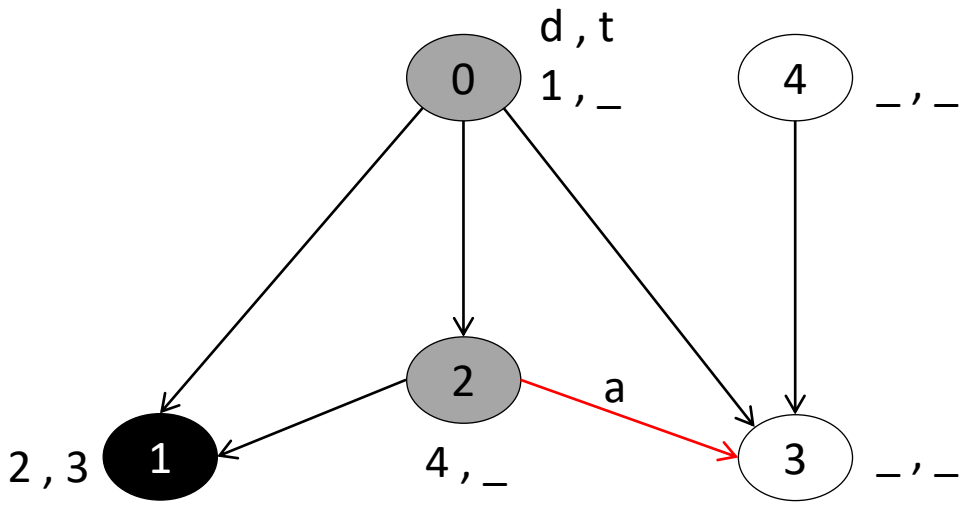


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 2
tempo = 4
cor [u] = cinza

a = aresta (2 -> 3)
v = 1

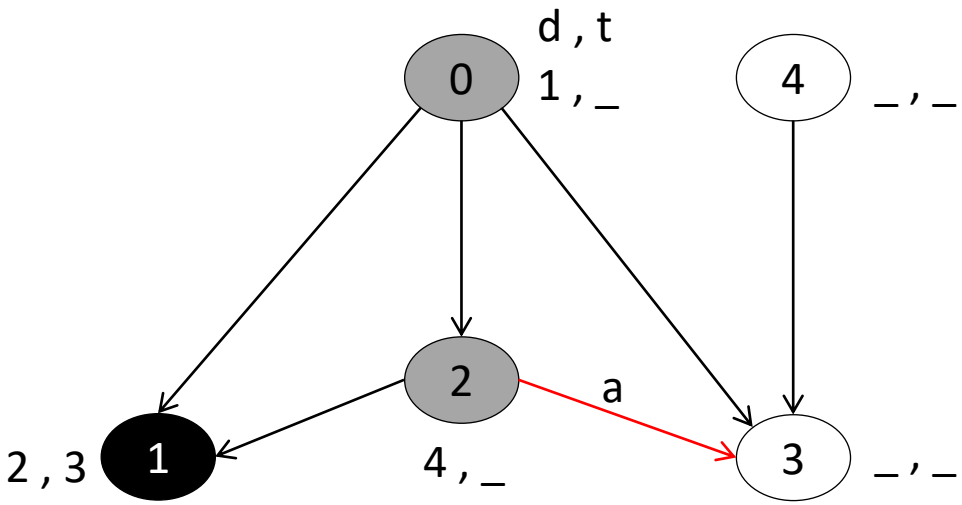


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 2
tempo = 4
cor [u] = cinza

a = aresta (2 -> 3)
v = 3

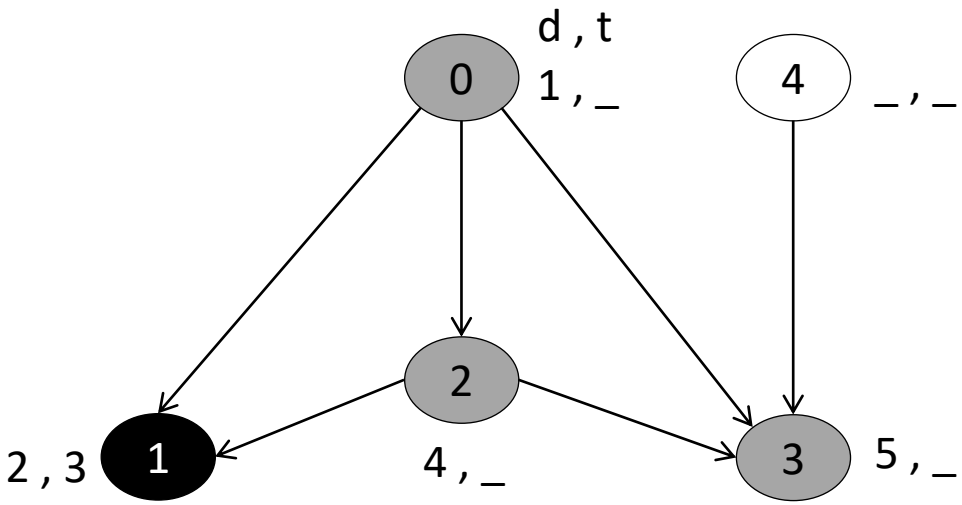


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 3
tempo = 5
cor [u] = cinza

a = aresta (2 -> 3)
v = 3

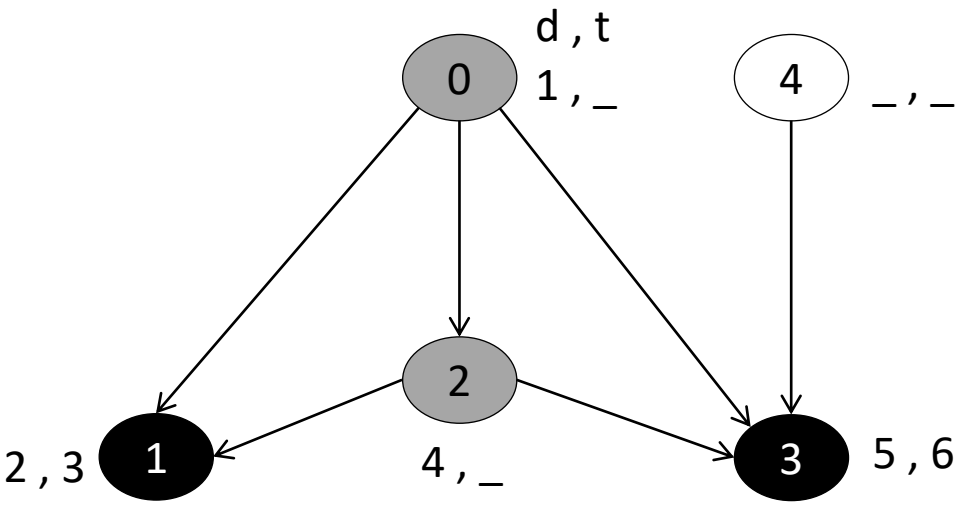


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 3
tempo = 6
cor [u] = preto

a = aresta (2 -> 3)
v = 3

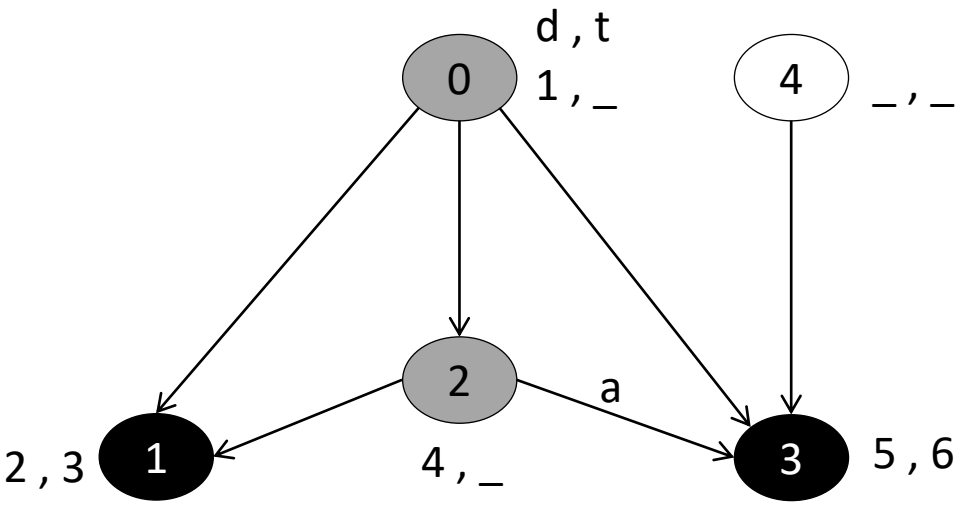


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 2
tempo = 6
cor [u] = cinza

a = aresta (2 -> 3)
v = 3

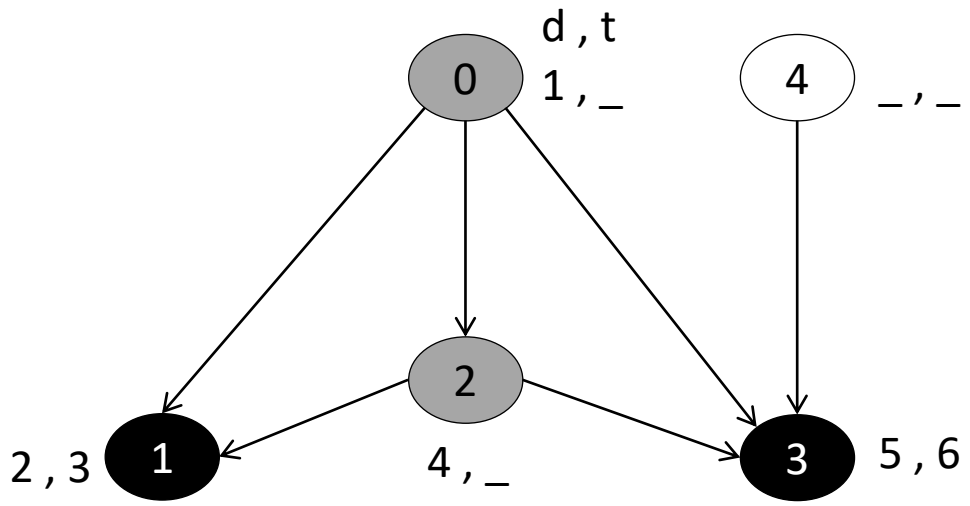


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 2
tempo = 6
cor [u] = cinza

a = NULL
v = 3

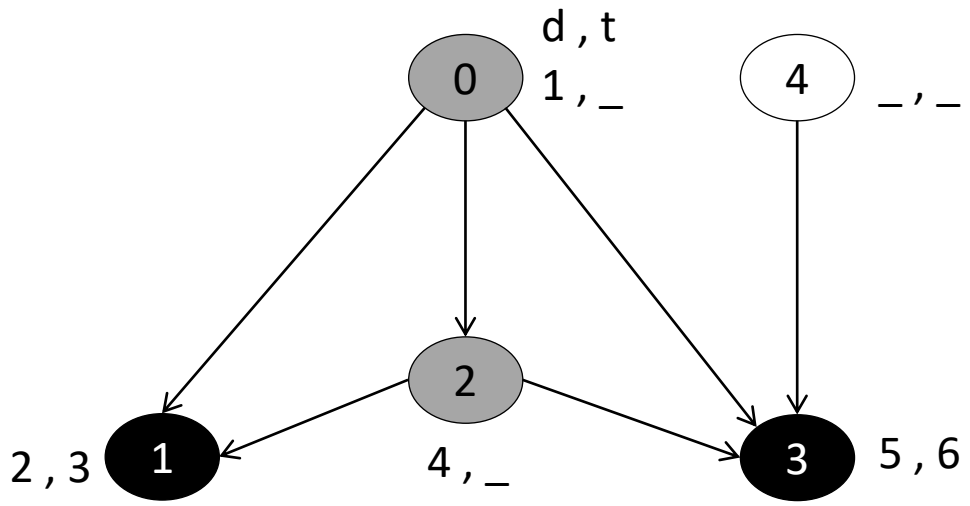


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 2
tempo = 6
cor [u] = cinza

a = NULL
v = 3

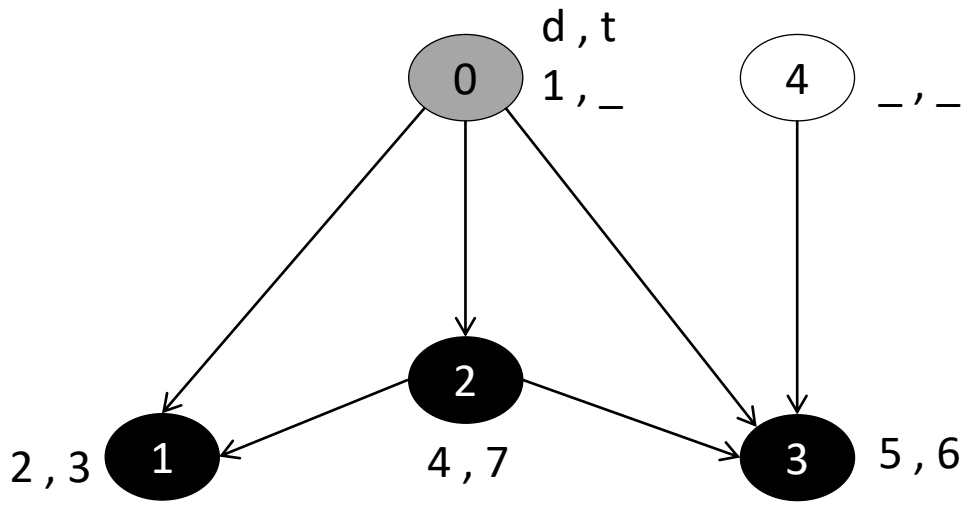


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 2
tempo = 7
cor [u] = preto

a = NULL
v = 3

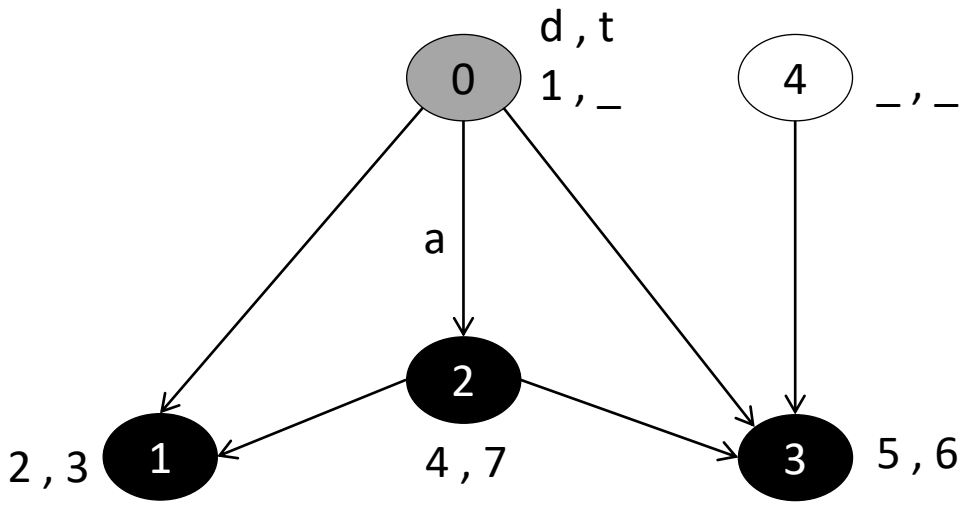


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 7
cor [u] = cinza

a = aresta (0 -> 2)
v = 2

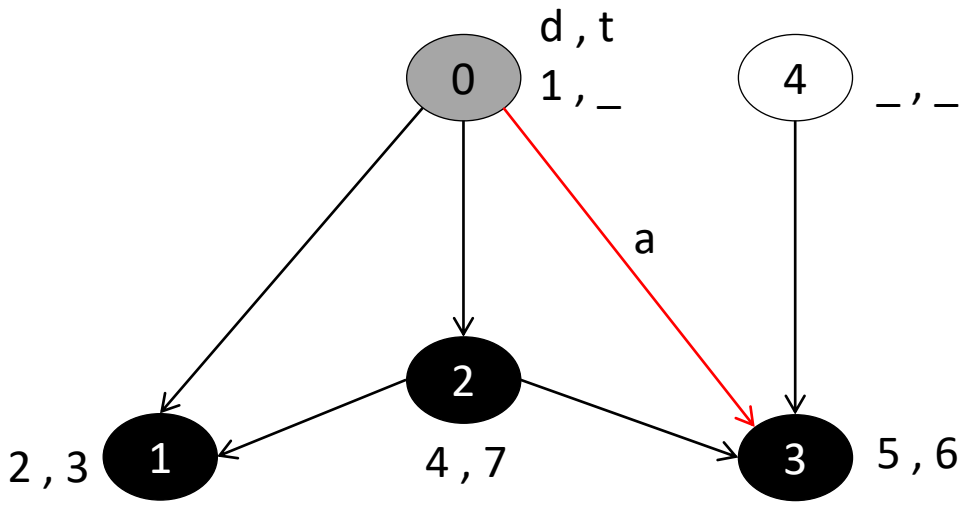


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 7
cor [u] = cinza

a = aresta (0 -> 3)
v = 2

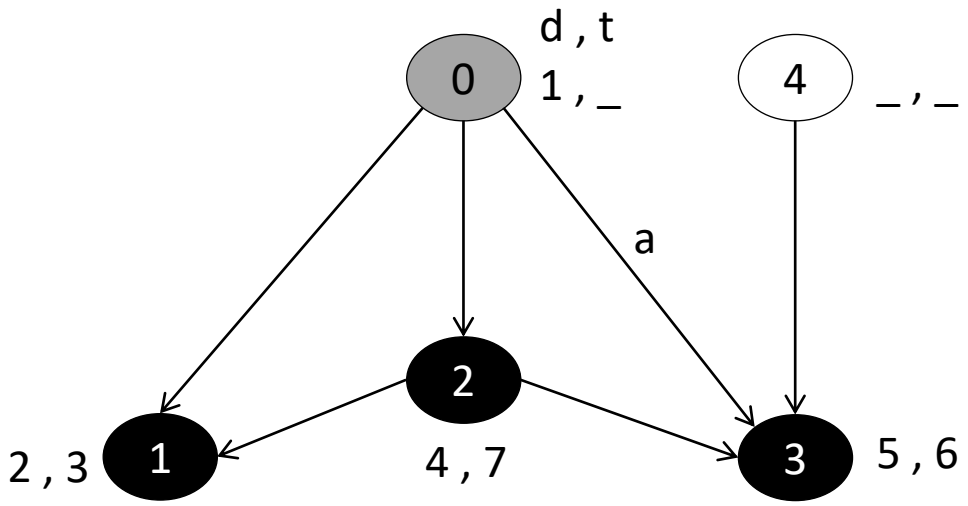


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 7
cor [u] = cinza

a = aresta (0 -> 3)
v = 3

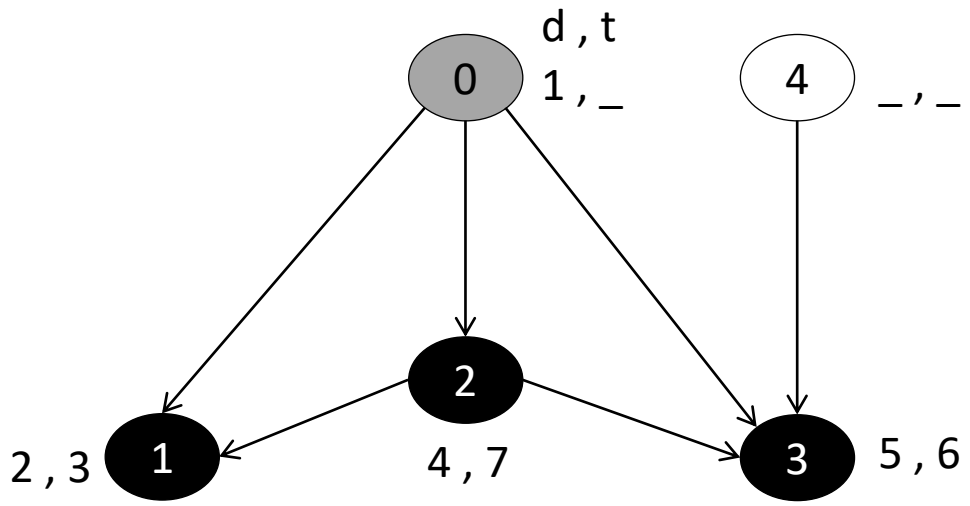


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 7
cor [u] = cinza

a = NULL
v = 3

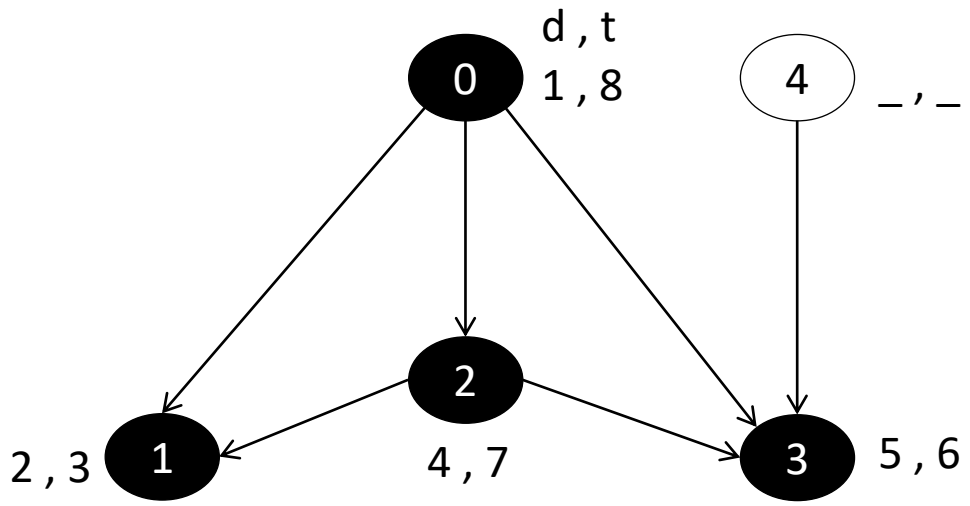


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 8
cor [u] = preto

a = NULL
v = 3

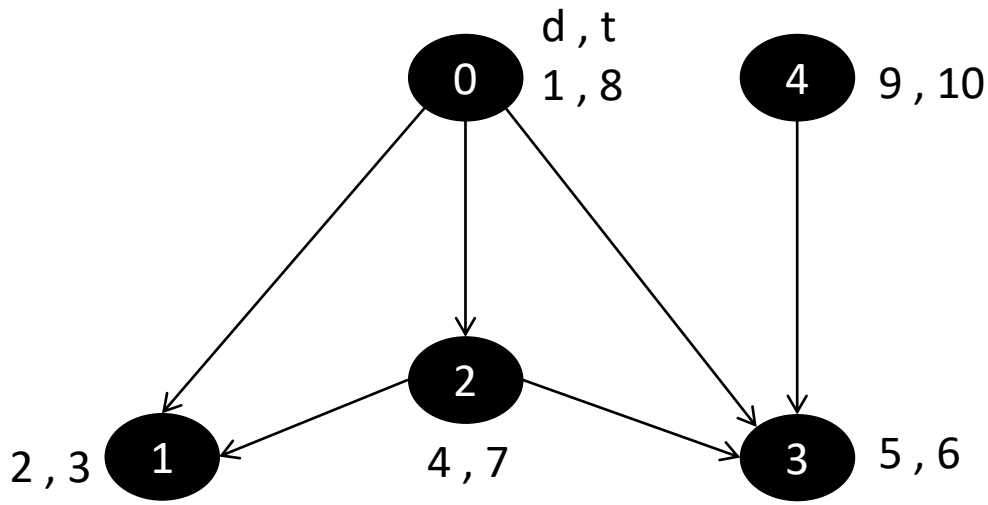


Busca em Profundidade

```
private int visitaDfs (int u, int tempo, int []cor)  {
    cor[u] = cinza; this.d[u] = ++tempo;

    if (!this.grafo.listaAdjVazia (u))
    {
        Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
         while (a != null)
         {
              int v = a. get_v2 ();
              if (cor[v] == branco)
              {
                 this.antecessor[v] = u;
                 tempo = this.visitaDfs (v, tempo, cor);
              }
              a = this.grafo.proxAdj (u);
         }
     }
     cor[u] = preto; this.t[u] = ++tempo;
     return tempo;
}
```

u = 0
tempo = 8
cor [u] = preto

a = NULL
v = 3



Neste passo, o mesmo procedimento é repetido para o vértice 4, até que a função seja finalizada.

Classificação das arestas no DFS

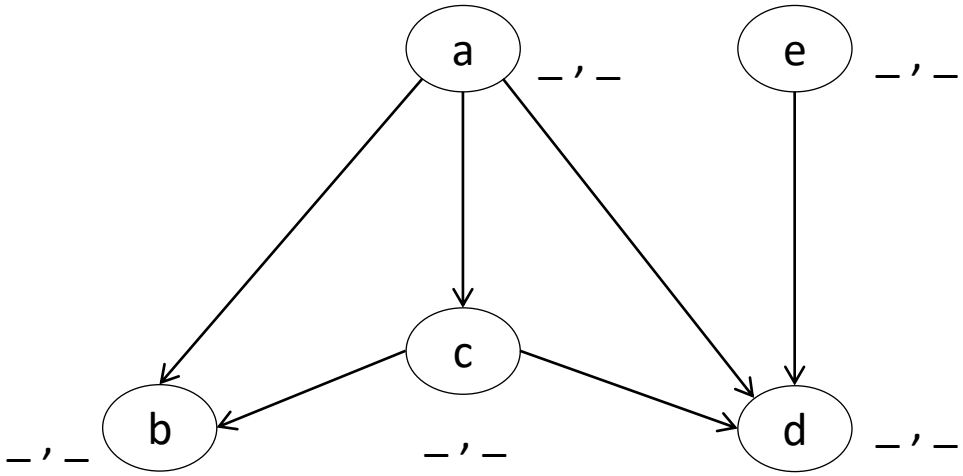
- **Arestas de árvore:** são arestas de uma árvore de busca em profundidade. A aresta (u, v) é uma aresta de árvore se v foi descoberto pela primeira vez ao percorrer a aresta (u, v). Ou seja, quando encontra um vértice branco.
- **Arestas de retorno**: conectam um vértice u com um antecessor v em uma árvore de busca em profundidade (inclui *self-loops*). De um descendente para um ancestral (Encontra um vértice cinza – sendo cinza para cinza).
- **Arestas de avanço**: ***não pertencem à árvore de busca em profundidade*** mas conectam um vértice a um descendente que pertence à árvore de busca em profundidade (de um ancestral para um descendente, quando encontra um vértice preto).
- **Arestas de cruzamento**: podem conectar vértices na mesma árvore de busca em profundidade, ou em duas árvores diferentes (de um vértice cinza para um vértice preto).

Classificação das arestas no DFS

- Classificação de arestas pode ser útil para derivar outros algoritmos.
- Na busca em profundidade cada aresta pode ser classificada pela cor do vértice que é alcançado pela primeira vez:
  - Branco indica uma aresta de árvore.
  - Cinza indica uma aresta de retorno.
  - Preto indica uma aresta de avanço quando u é descoberto antes de v ou uma aresta de cruzamento caso contrário.

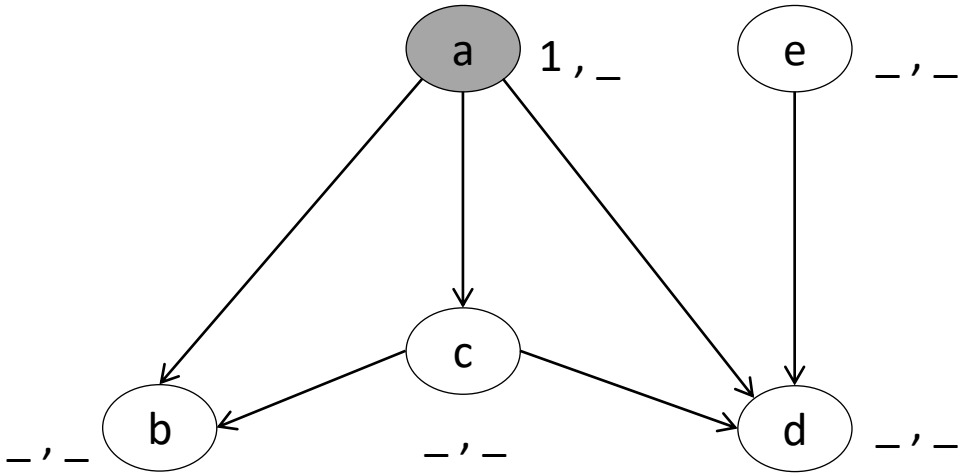
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



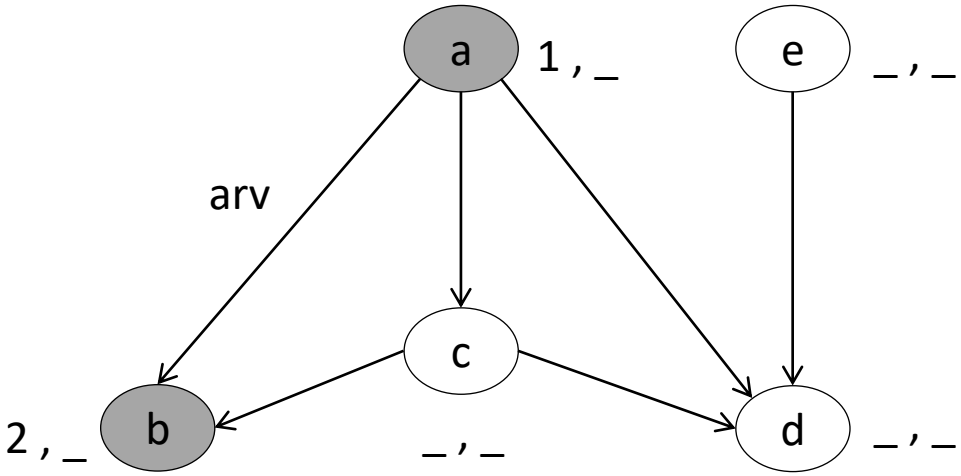
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



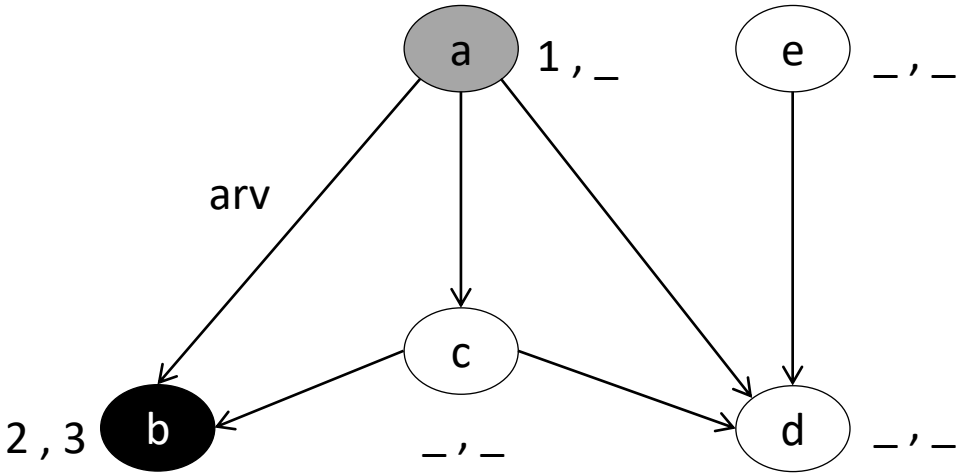
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



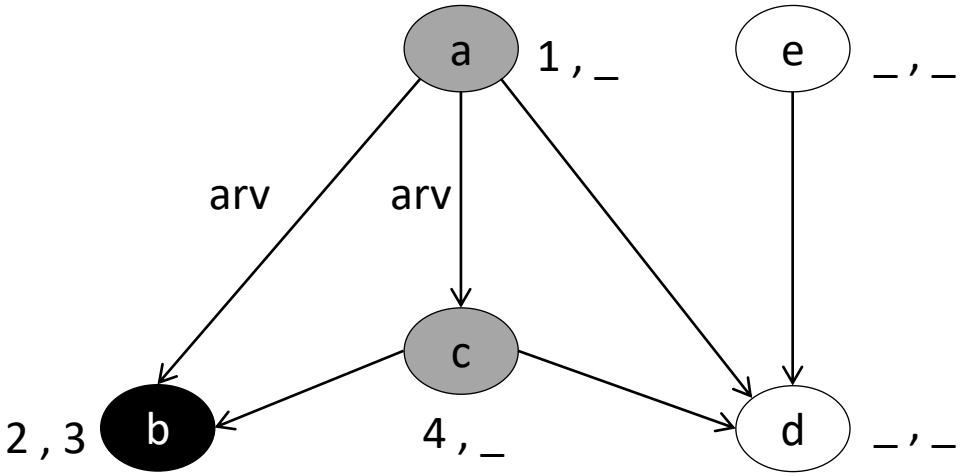
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



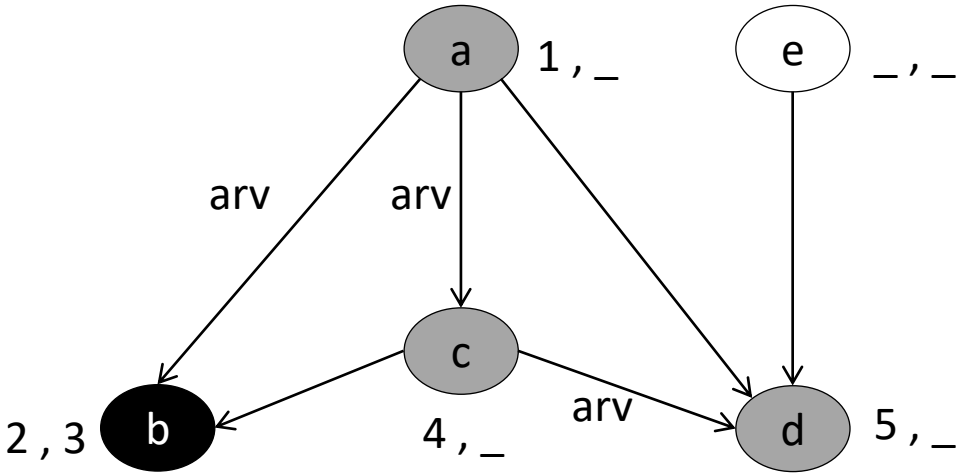
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



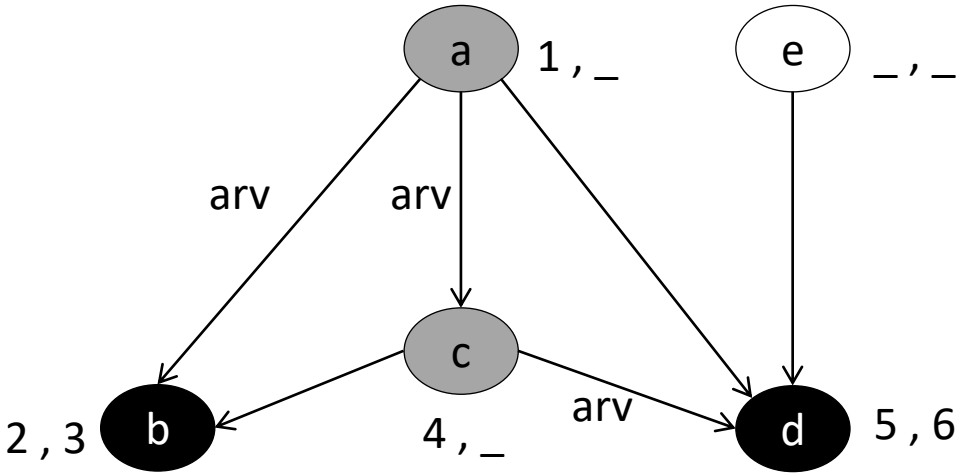
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



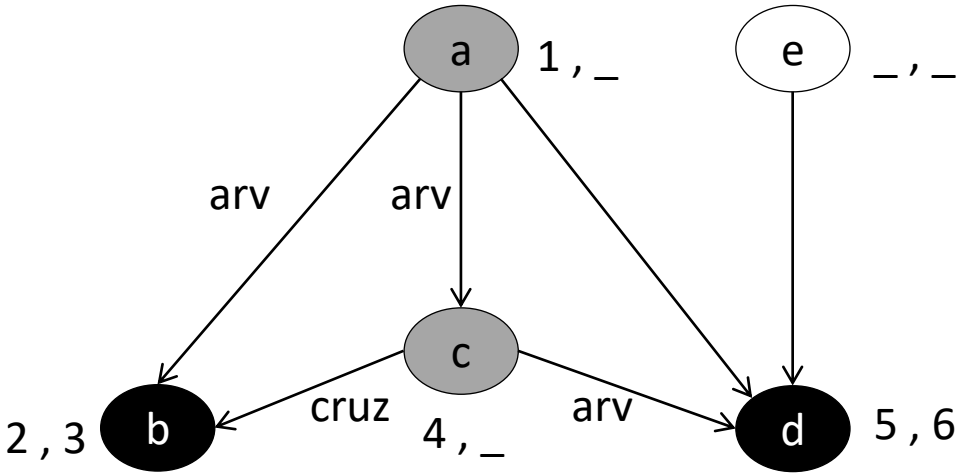
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



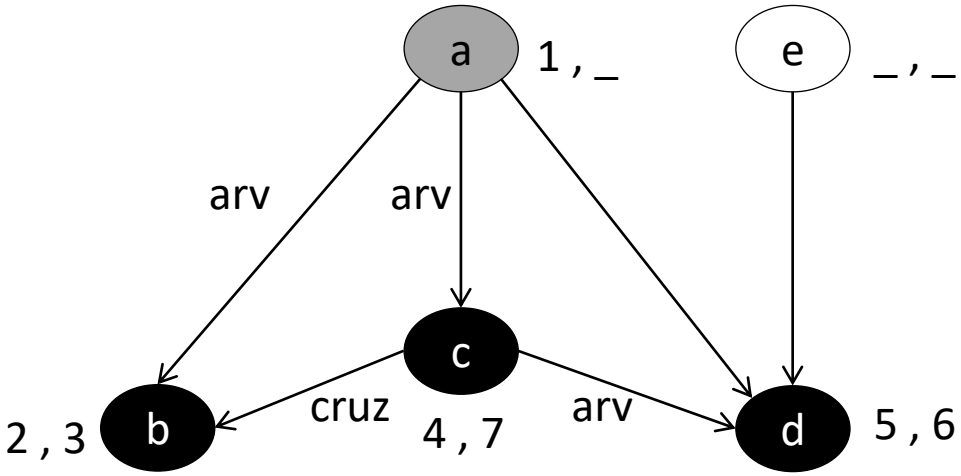
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



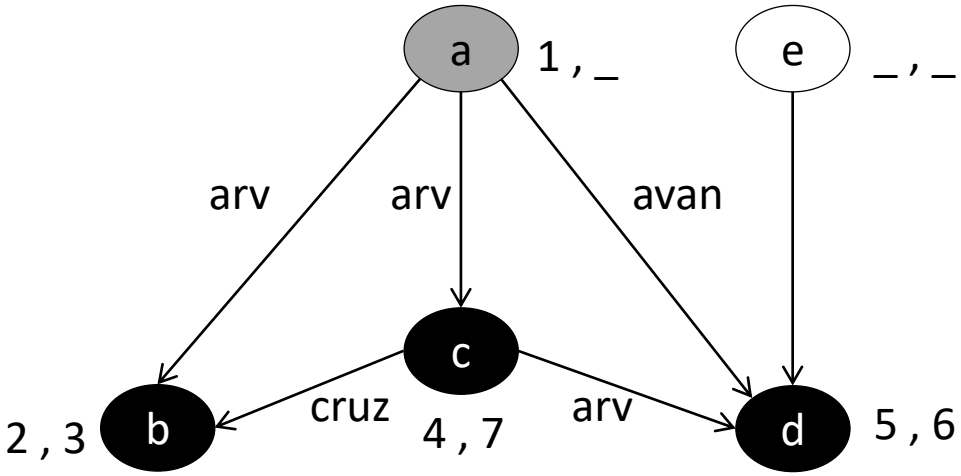
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



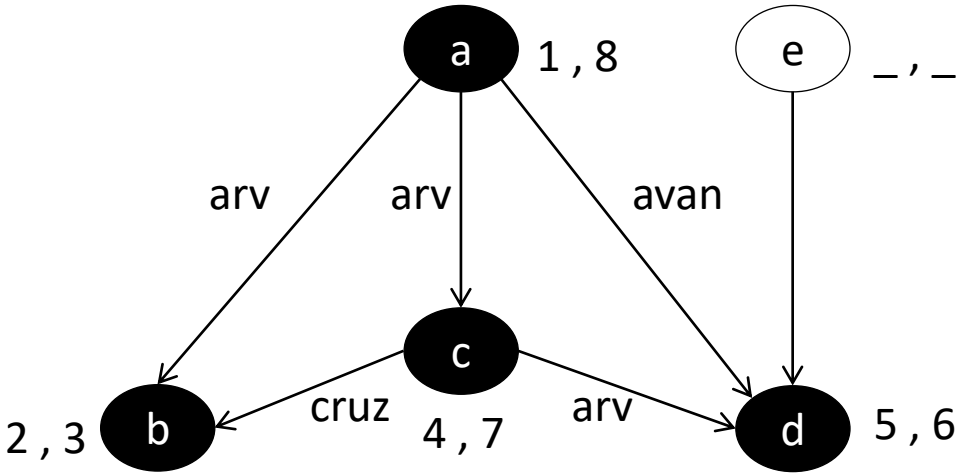
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



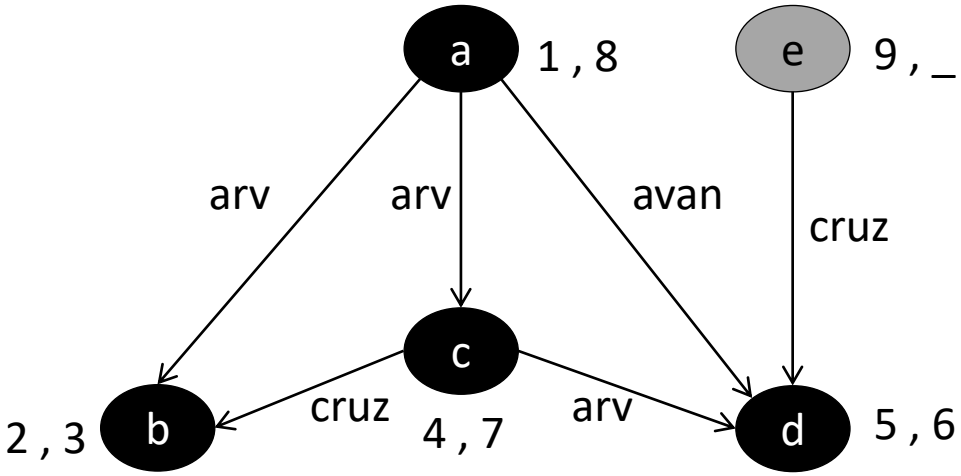
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



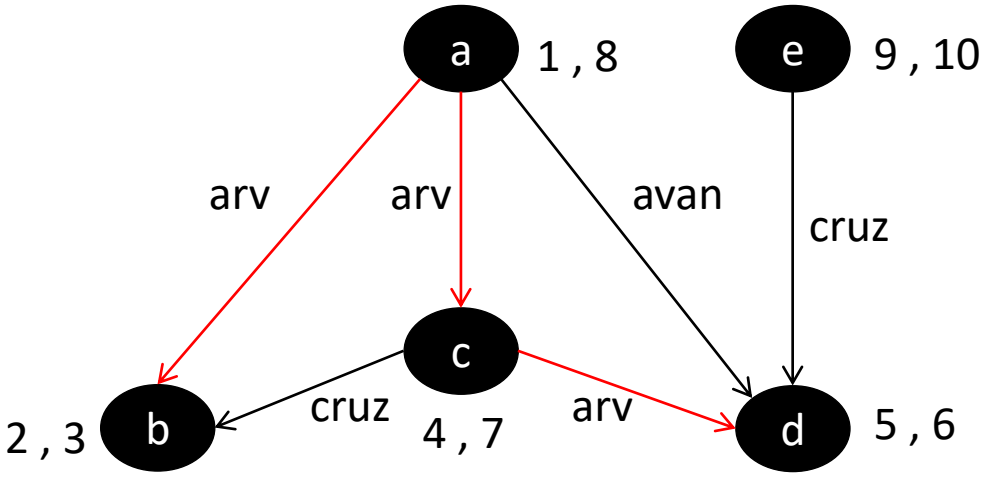
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



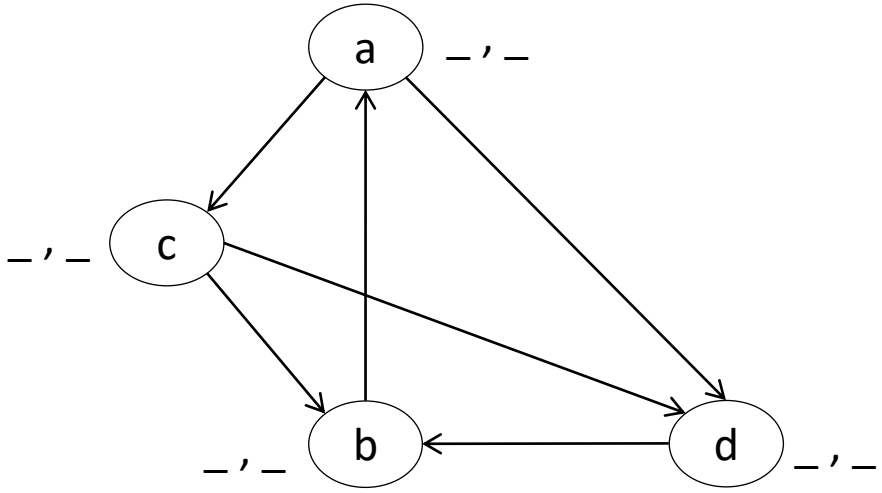
Busca em Profundidade

- **Arestas de árvore:** encontra um vértice branco.
- **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).
- **Arestas de avanço**: encontra um vértice preto (u é descoberto antes de v).
- **Arestas de cruzamento**: de um vértice cinza para um vértice preto.



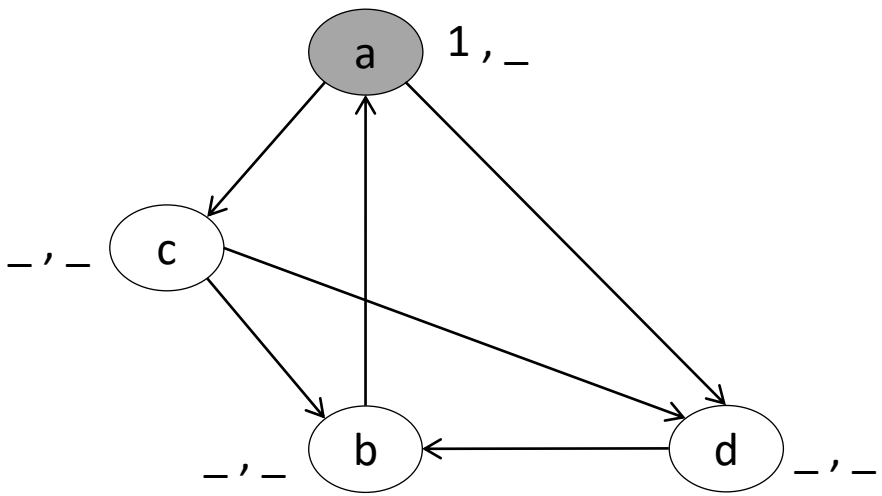
Busca em Profundidade

- Exemplo de uma aresta de retorno:
  - **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).



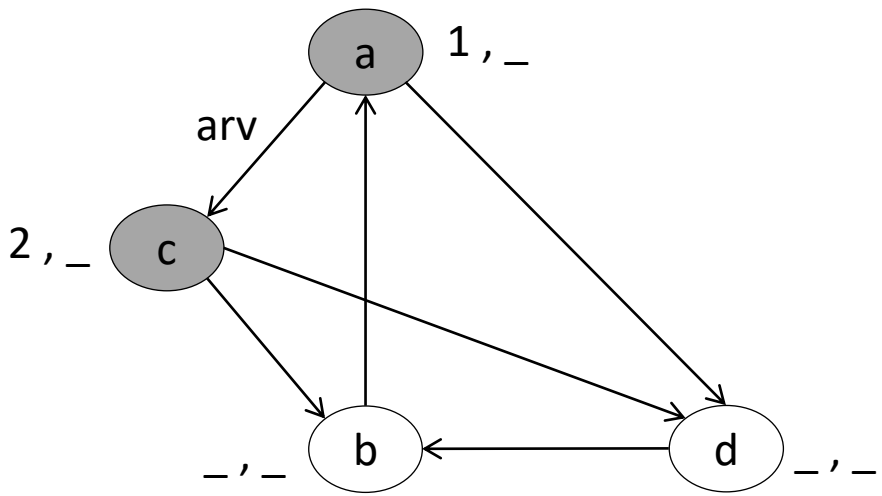
Busca em Profundidade

- Exemplo de uma aresta de retorno:
  - **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).



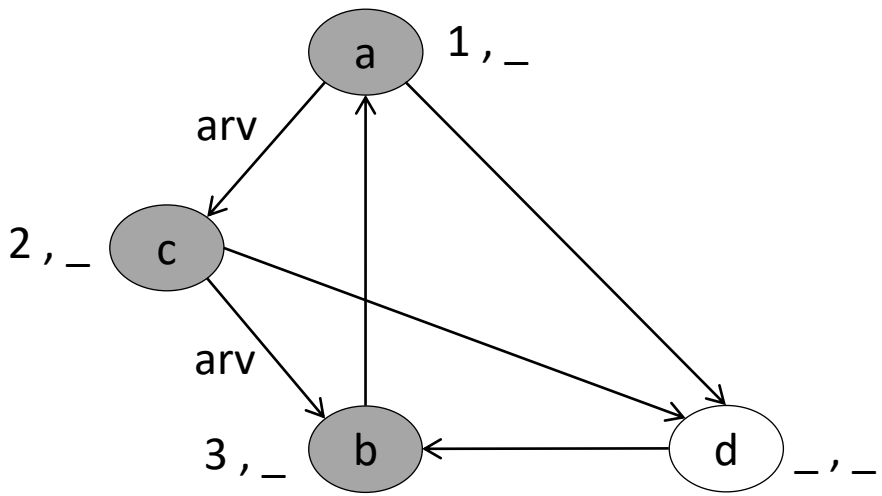
Busca em Profundidade

- Exemplo de uma aresta de retorno:
  - **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).



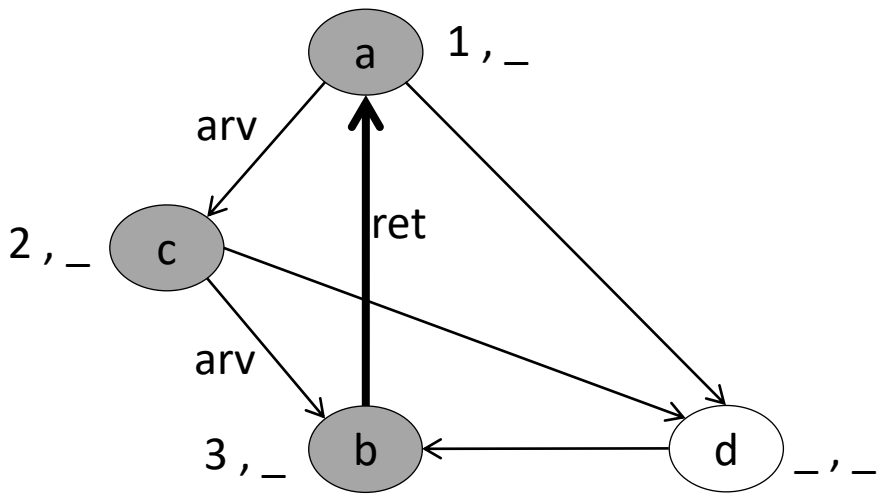
Busca em Profundidade

- Exemplo de uma aresta de retorno:
  - **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).



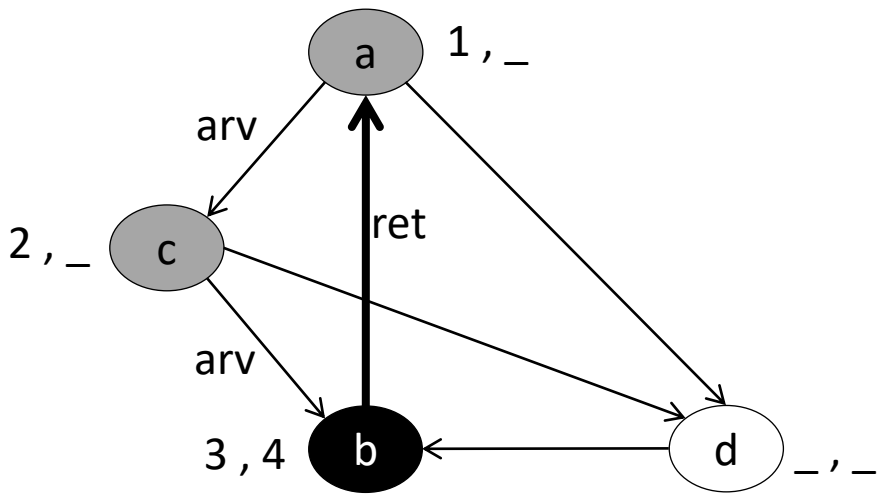
Busca em Profundidade

- Exemplo de uma aresta de retorno:
  - **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).



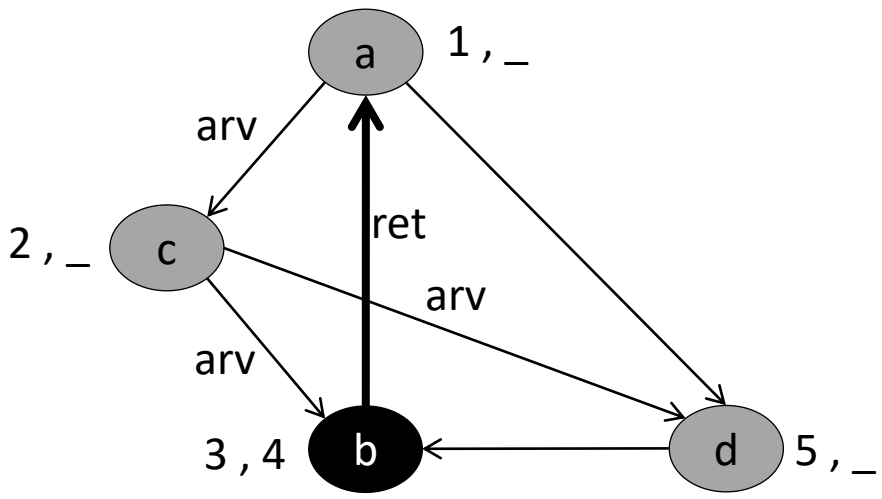
Busca em Profundidade

- Exemplo de uma aresta de retorno:
  - **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).



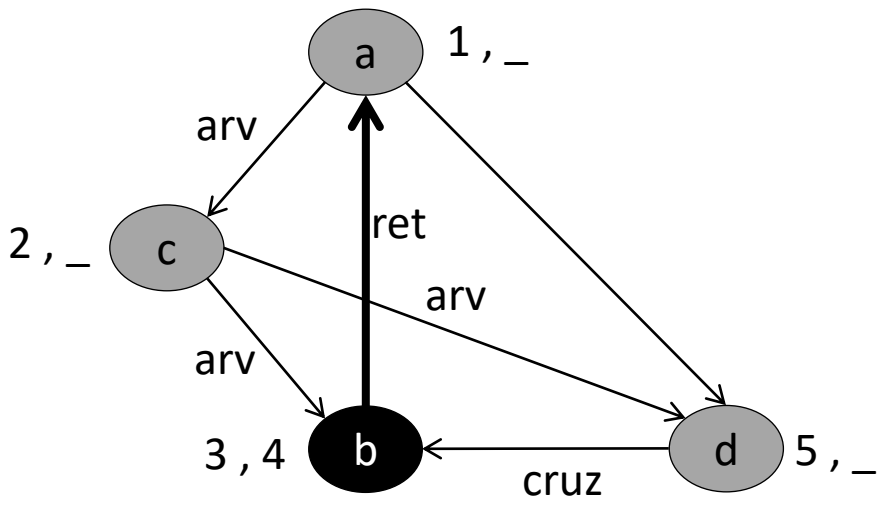
Busca em Profundidade

- Exemplo de uma aresta de retorno:
  - **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).



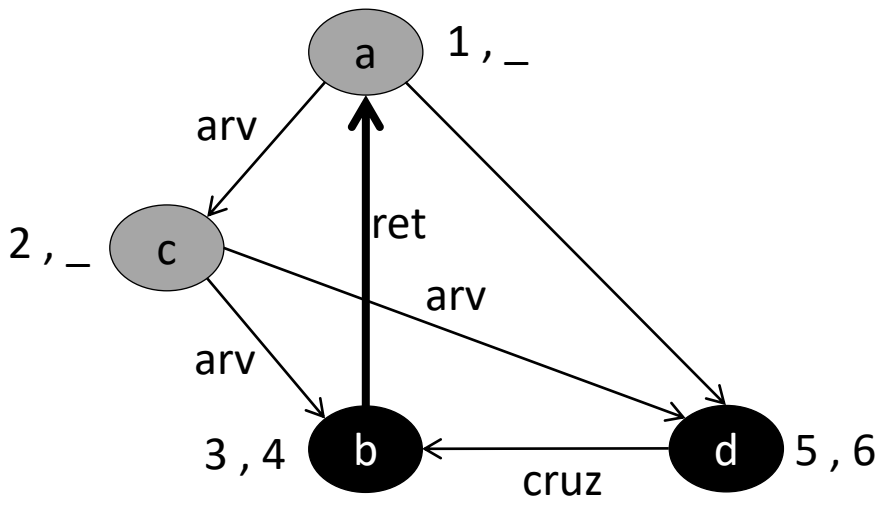
Busca em Profundidade

- Exemplo de uma aresta de retorno:
  - **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).



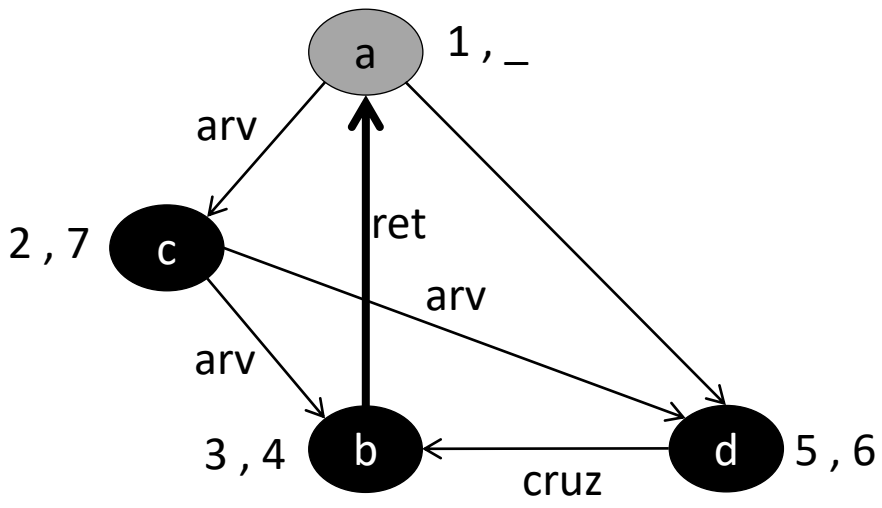
Busca em Profundidade

- Exemplo de uma aresta de retorno:
  - **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).



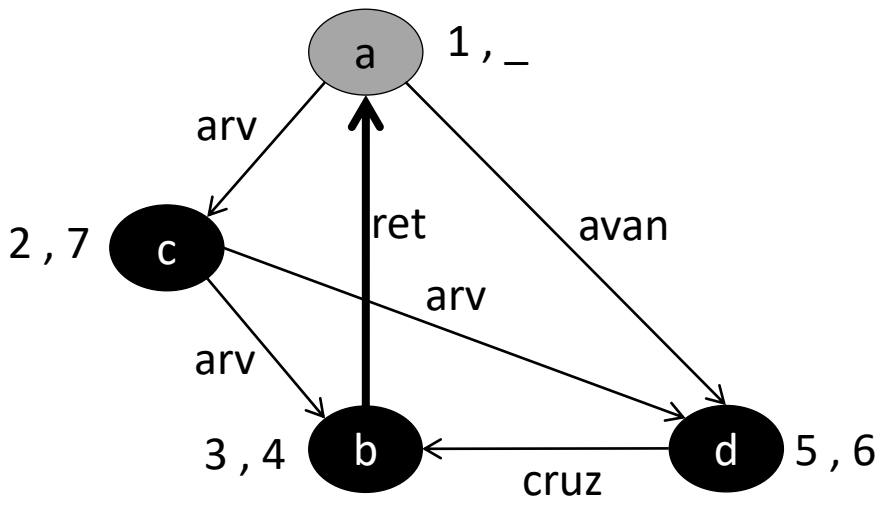
Busca em Profundidade

- Exemplo de uma aresta de retorno:
  - **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).



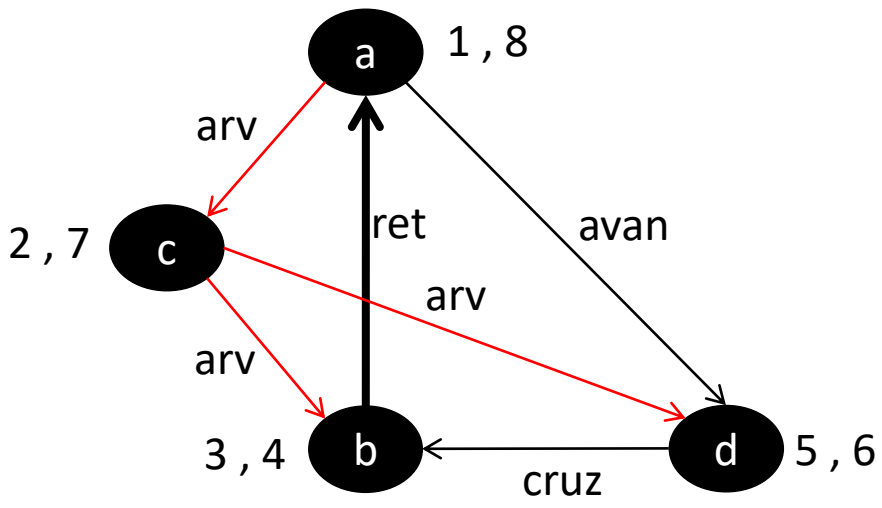
Busca em Profundidade

- Exemplo de uma aresta de retorno:
  - **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).



Busca em Profundidade

- Exemplo de uma aresta de retorno:
  - **Arestas de retorno**: Encontra um vértice cinza (cinza para cinza).



Análise da Busca em Profundidade

- O método $visitaDfs$ é chamado exatamente uma vez para cada vértice $u \in V$.
- Durante a execução de $visitaDfs(u, tempo, cor)$, o anel principal é executado $|adj[u]|$ vezes (ou seja, para cada aresta de u).
- Então:

$$\sum_{u \in V} |adj[u]| = O(|A|)$$

- O tempo total de execução de $visitaDfs$ é $O(|A|)$.
- Logo, a complexidade do algoritmo é $O(|V| + |A|)$.

Busca em Profundidade

- Teste de circuito
  - Se uma **aresta de retorno** é encontrada na busca em profundidade então o grafo possui um **ciclo**
  - Um grafo é acíclico se e somente se na busca em profundidade não for encontrada nenhuma **aresta de retorno**.

Busca em largura

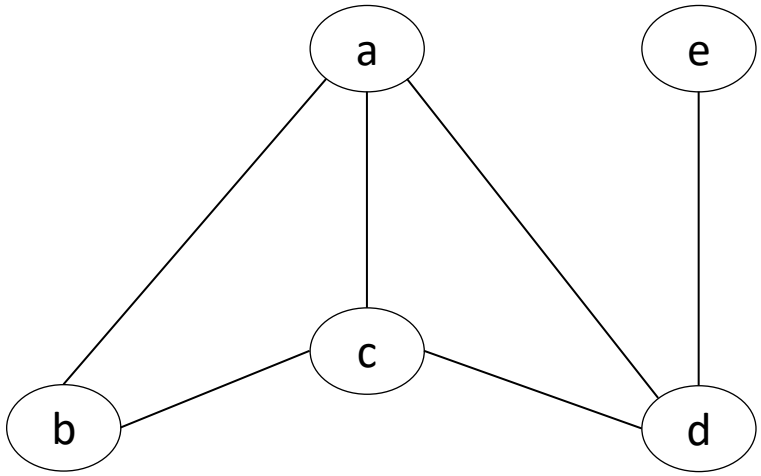
- Expandir o conjunto de vértices de forma uniforme em que são visitados todos os vértices de mesma distância ao início antes de visitar outros níveis.
- Ideia
  - Na busca em largura o algoritmo descobre todos os vértices a uma distância k do vértice de origem antes de descobrir os que estão a uma distância $k + 1$.
  - O grafo pode ser direcionado ou não direcionado

Busca em largura

- Algoritmo:
  - Cada vértice é colorido de branco, cinza ou preto.
  - Todos os vértices são inicializados com branco
  - Quando um vértice é descoberto pela primeira vez ele torna-se cinza
  - Vértices cujos adjacentes são todos descobertos tornam-se pretos.
  - Se $(u, v) \in A$ e o vértice u é preto, então v tem quer ser cinza ou preto.
  - Vértices cinzas podem ter adjacentes brancos.

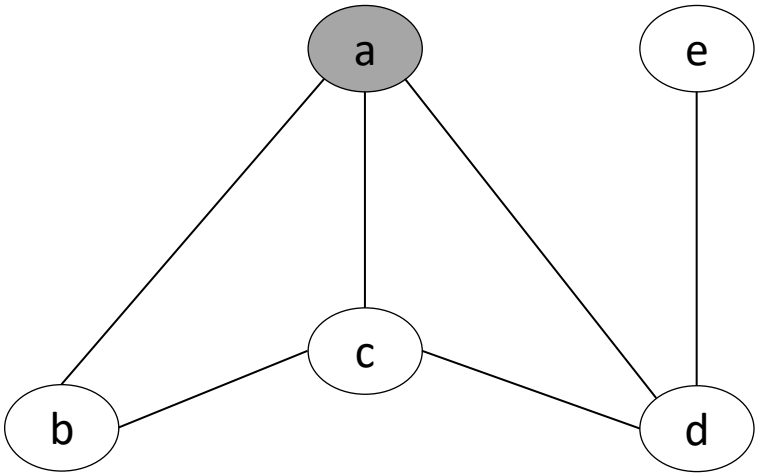
Busca em Largura

- Algoritmo:
  - Todos os vértices são inicializados com branco
  - Quando um vértice é descoberto pela primeira vez ele torna-se cinza
  - Vértices cujos adjacentes são todos descobertos tornam-se pretos.
  - Se $(u, v) \in A$ e o vértice u é preto, então v tem quer ser cinza ou preto.



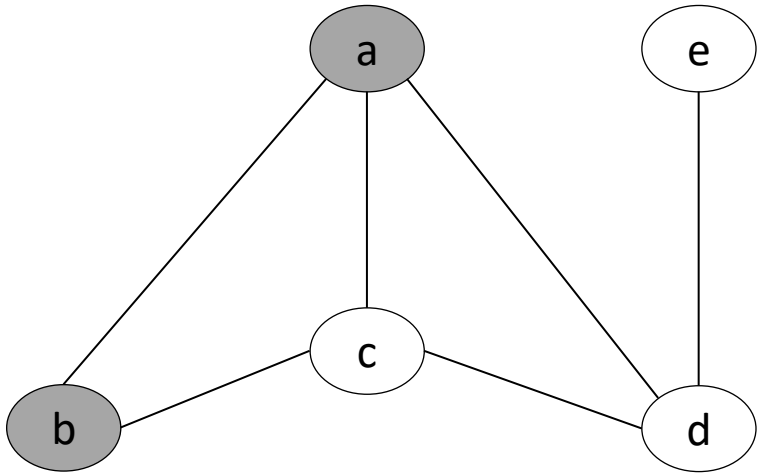
Busca em Largura

- Algoritmo:
  - Todos os vértices são inicializados com branco
  - Quando um vértice é descoberto pela primeira vez ele torna-se cinza
  - Vértices cujos adjacentes são todos descobertos tornam-se pretos.
  - Se $(u, v) \in A$ e o vértice u é preto, então v tem quer ser cinza ou preto.



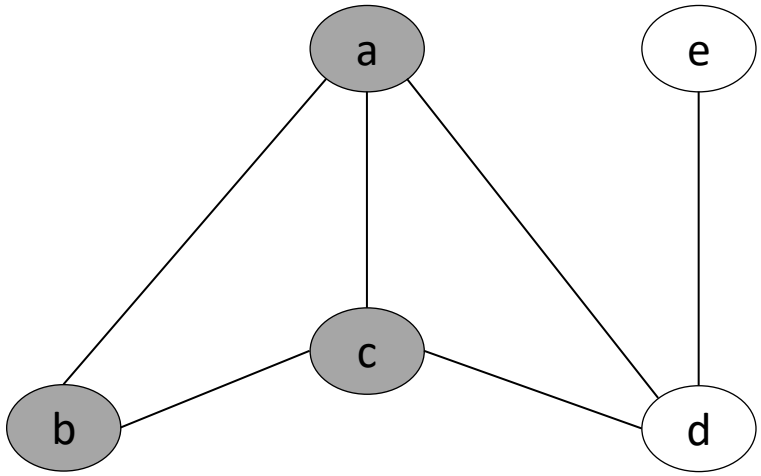
Busca em Largura

- Algoritmo:
  - Todos os vértices são inicializados com branco
  - Quando um vértice é descoberto pela primeira vez ele torna-se cinza
  - Vértices cujos adjacentes são todos descobertos tornam-se pretos.
  - Se $(u, v) \in A$ e o vértice u é preto, então v tem quer ser cinza ou preto.



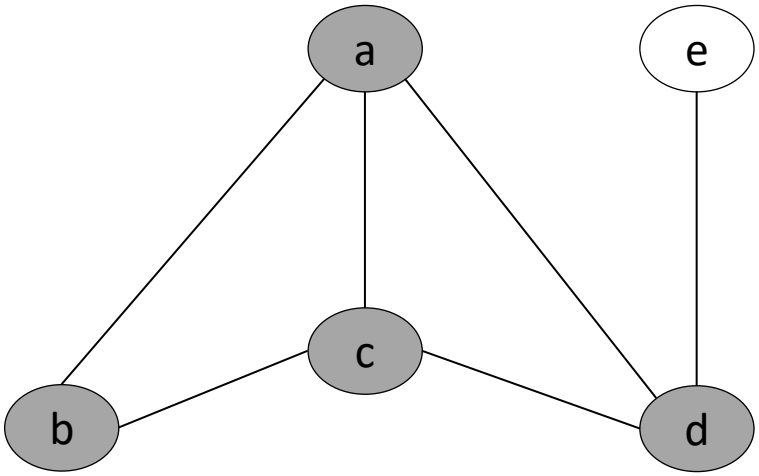
Busca em Largura

- Algoritmo:
  - Todos os vértices são inicializados com branco
  - Quando um vértice é descoberto pela primeira vez ele torna-se cinza
  - Vértices cujos adjacentes são todos descobertos tornam-se pretos.
  - Se $(u, v) \in A$ e o vértice u é preto, então v tem quer ser cinza ou preto.



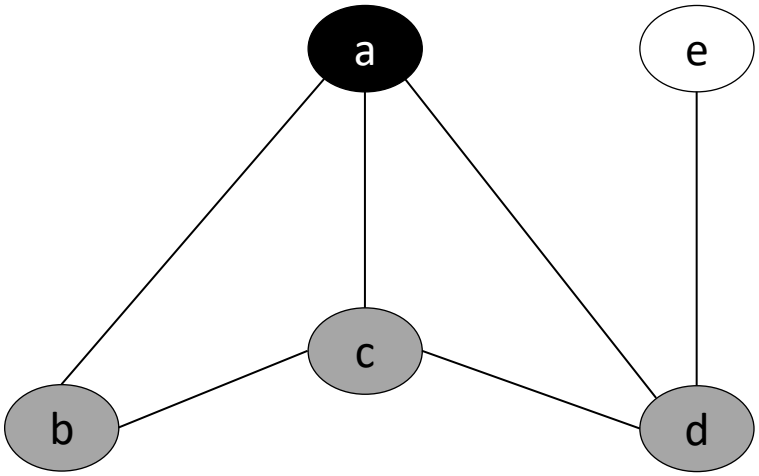
Busca em Largura

- Algoritmo:
  - Todos os vértices são inicializados com branco
  - Quando um vértice é descoberto pela primeira vez ele torna-se cinza
  - Vértices cujos adjacentes são todos descobertos tornam-se pretos.
  - Se $(u, v) \in A$ e o vértice u é preto, então v tem quer ser cinza ou preto.



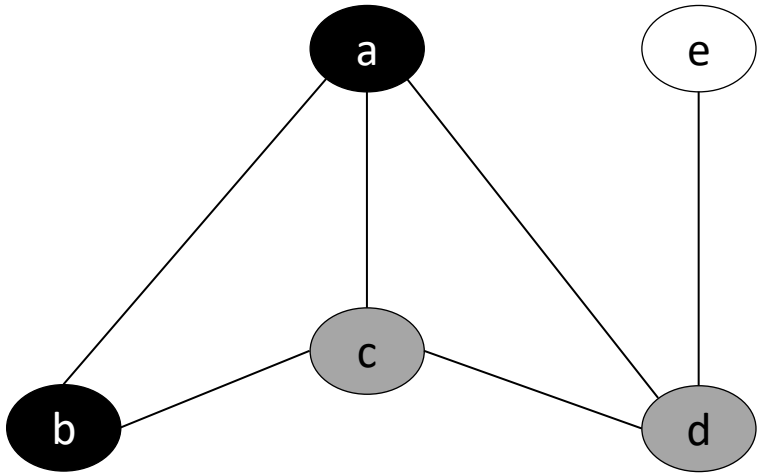
Busca em Largura

- Algoritmo:
  - Todos os vértices são inicializados com branco
  - Quando um vértice é descoberto pela primeira vez ele torna-se cinza
  - Vértices cujos adjacentes são todos descobertos tornam-se pretos.
  - Se $(u, v) \in A$ e o vértice u é preto, então v tem quer ser cinza ou preto.



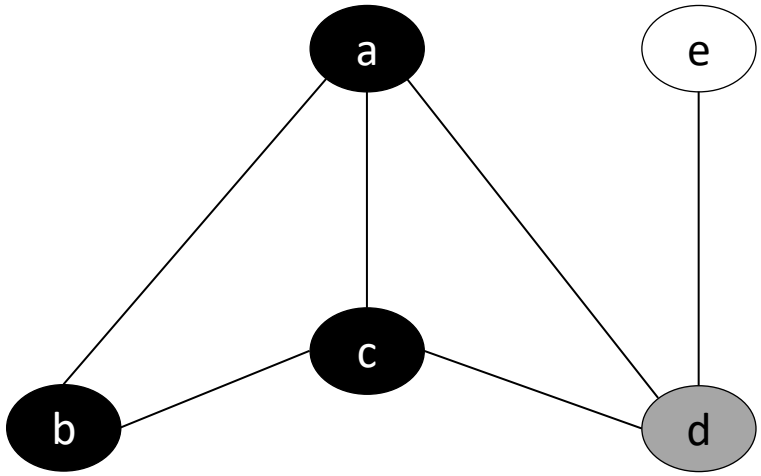
Busca em Largura

- Algoritmo:
  - Todos os vértices são inicializados com branco
  - Quando um vértice é descoberto pela primeira vez ele torna-se cinza
  - Vértices cujos adjacentes são todos descobertos tornam-se pretos.
  - Se $(u, v) \in A$ e o vértice u é preto, então v tem quer ser cinza ou preto.



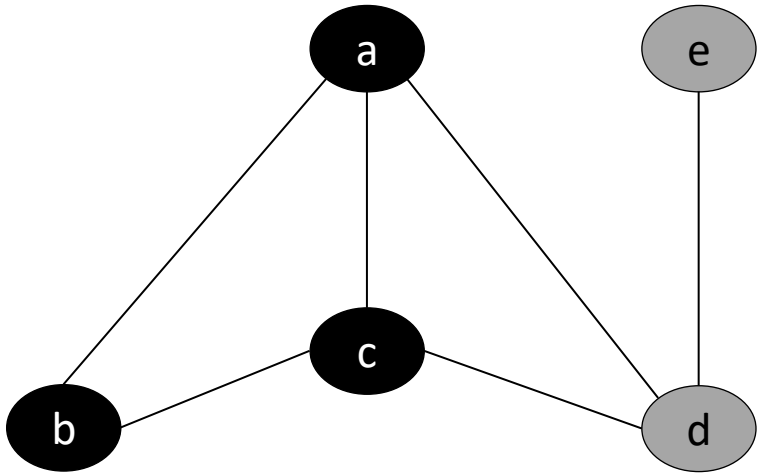
Busca em Largura

- Algoritmo:
  - Todos os vértices são inicializados com branco
  - Quando um vértice é descoberto pela primeira vez ele torna-se cinza
  - Vértices cujos adjacentes são todos descobertos tornam-se pretos.
  - Se $(u, v) \in A$ e o vértice u é preto, então v tem quer ser cinza ou preto.



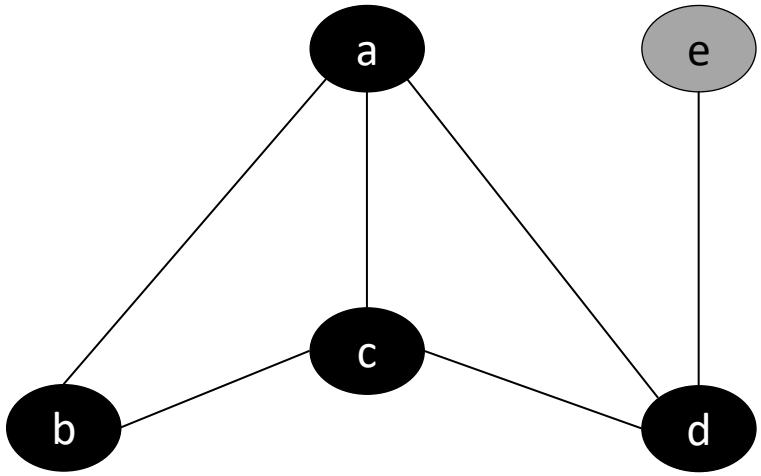
Busca em Largura

- Algoritmo:
  - Todos os vértices são inicializados com branco
  - Quando um vértice é descoberto pela primeira vez ele torna-se cinza
  - Vértices cujos adjacentes são todos descobertos tornam-se pretos.
  - Se $(u, v) \in A$ e o vértice u é preto, então v tem quer ser cinza ou preto.



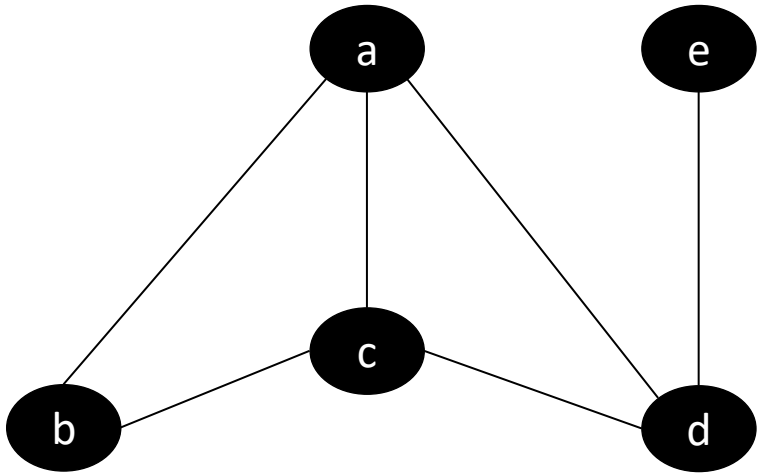
Busca em Largura

- Algoritmo:
  - Todos os vértices são inicializados com branco
  - Quando um vértice é descoberto pela primeira vez ele torna-se cinza
  - Vértices cujos adjacentes são todos descobertos tornam-se pretos.
  - Se $(u, v) \in A$ e o vértice u é preto, então v tem quer ser cinza ou preto.



Busca em Largura

- Algoritmo:
  - Todos os vértices são inicializados com branco
  - Quando um vértice é descoberto pela primeira vez ele torna-se cinza
  - Vértices cujos adjacentes são todos descobertos tornam-se pretos.
  - Se $(u, v) \in A$ e o vértice u é preto, então v tem quer ser cinza ou preto.



Busca em Largura

```
public class BuscaEmLargura    {
      public const byte branco = 0;
      public const byte cinza  = 1;
      public const byte preto  = 2;
      private int []d;
      private int [] antecessor;
      private Grafo grafo;

      public BuscaEmLargura (Grafo grafo) {
         this.grafo = grafo; int n = this.grafo.get_numVertices();
         this.d = new int[n]; this.antecessor = new int[n];
      }
      public int get_d (int v) { return this.d[v]; }
      public int get_antecessor(int v) { return this.antecessor[v]; }

      public void imprimeCaminho (int origem, int v) {
         if (origem == v)
            Console.WriteLine (origem);
         else if (this.antecessor[v] == -1)
            Console.WriteLine ("Nao existe caminho de " + origem + " ate " + v);
         else {
            imprimeCaminho (origem, this.antecessor[v]);
            Console.WriteLine (v);
         }
      }
```

Busca em Largura

```
public void buscaEmLargura () {
        int []cor = new int[this.grafo.get_numVertices ()];

        for (int u = 0; u < grafo.get_numVertices(); u++)
        {
           cor[u] = branco; this.d[u] = Int32.MaxValue;
           this.antecessor[u] = -1;
        }
        for (int u = 0; u < grafo.get_numVertices(); u++)
        if (cor[u] == branco)
                this.visitaBfs (u, cor);
    }
```

Busca em Largura

```
public void buscaEmLargura () {
        int []cor = new int[this.grafo.get_numVertices ()];

        for (int u = 0; u < grafo.get_numVertices(); u++)
        {
           cor[u] = branco; this.d[u] = Int32.MaxValue;
           this.antecessor[u] = -1;
        }
        for (int u = 0; u < grafo.get_numVertices(); u++)
        if (cor[u] == branco)
                this.visitaBfs (u, cor);
    }
```

Início da chamada da
função visitaBfs

Busca em Largura

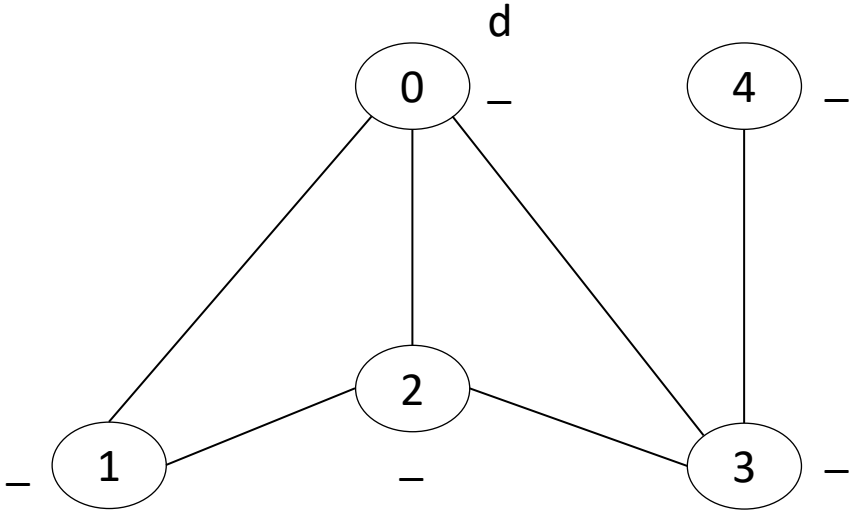
vértice atual → u = 0

Fila:

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a =

v =



Busca em Largura

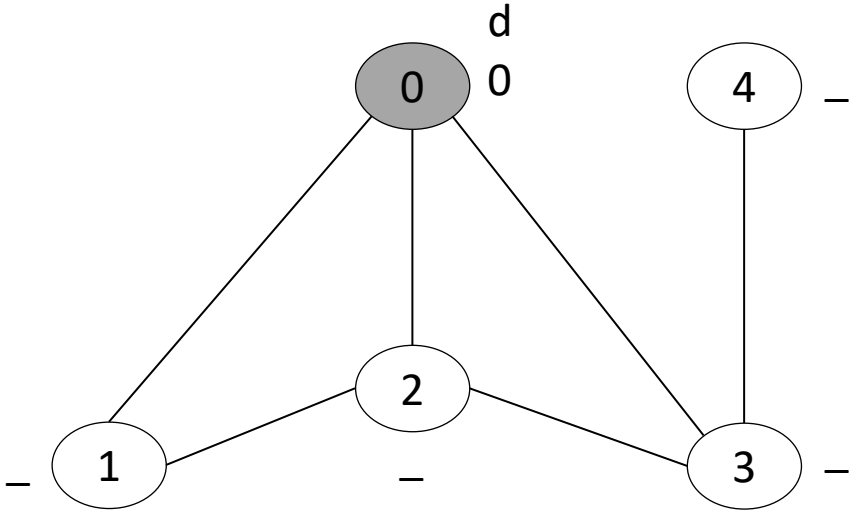
vértice atual → u = 0

Fila:

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a =

v =



Busca em Largura

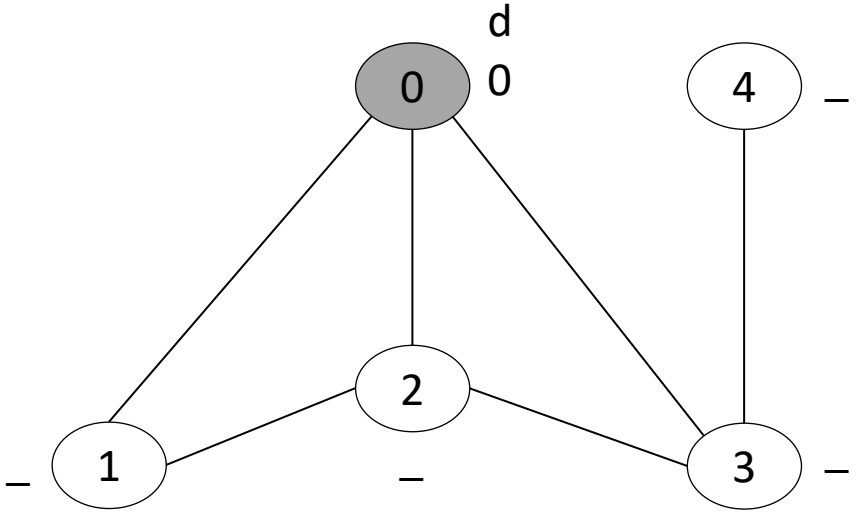
vértice atual → u = 0

Fila: 0 -

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
          {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a =

v =



Busca em Largura

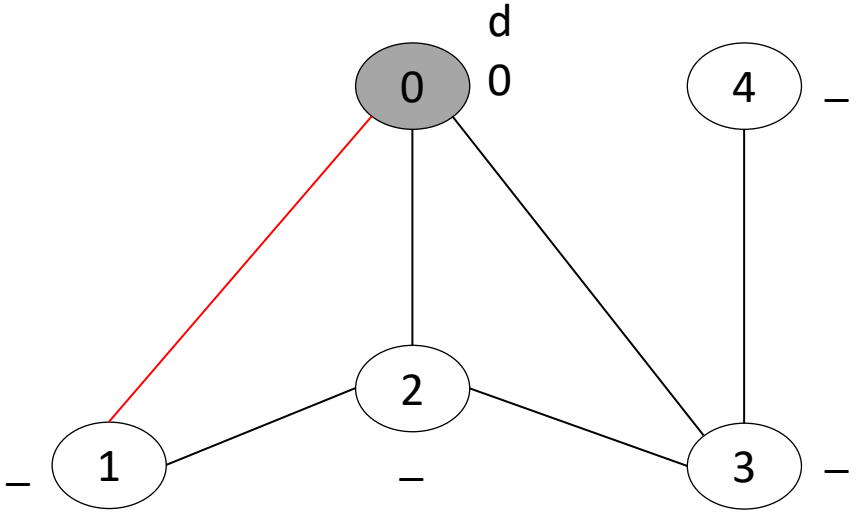
vértice atual → u = 0

Fila:

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = (0 - 1)

v =



Busca em Largura

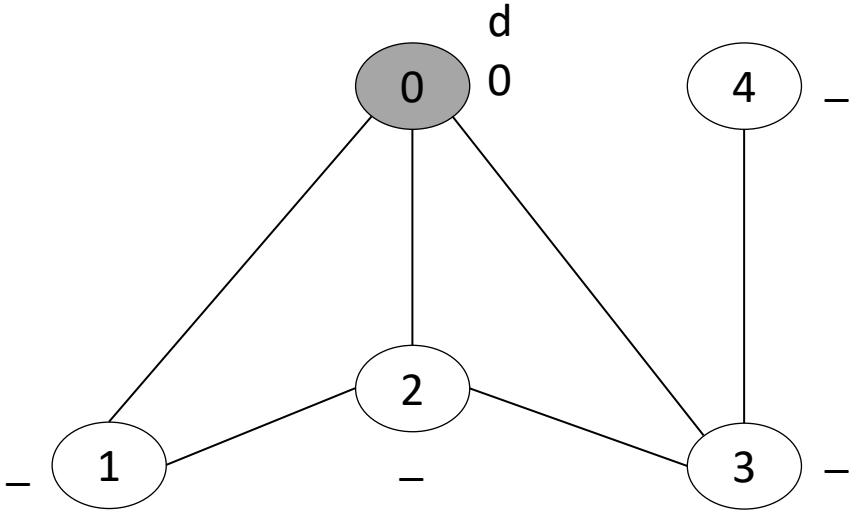
vértice atual → u = 0

Fila:

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = (0 - 1)

v = 1



Busca em Largura

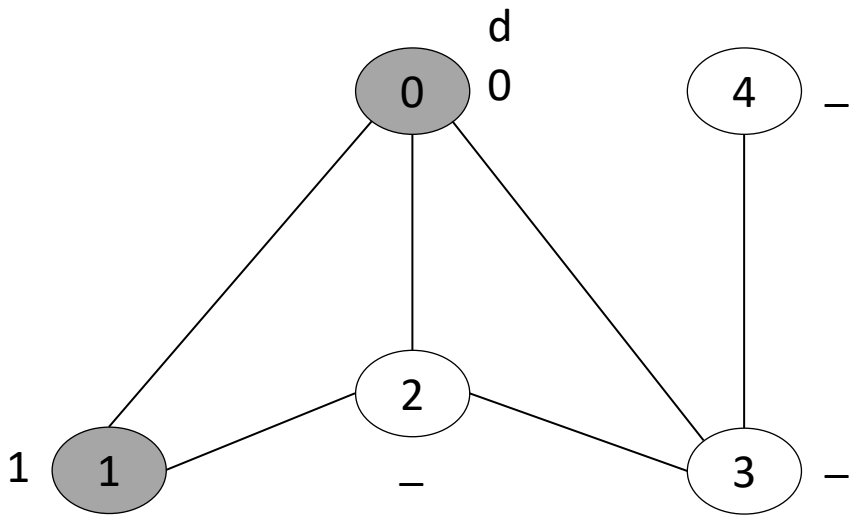
vértice atual → u = 0

Fila: 1 -

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = (0 - 1)

v = 1



Busca em Largura

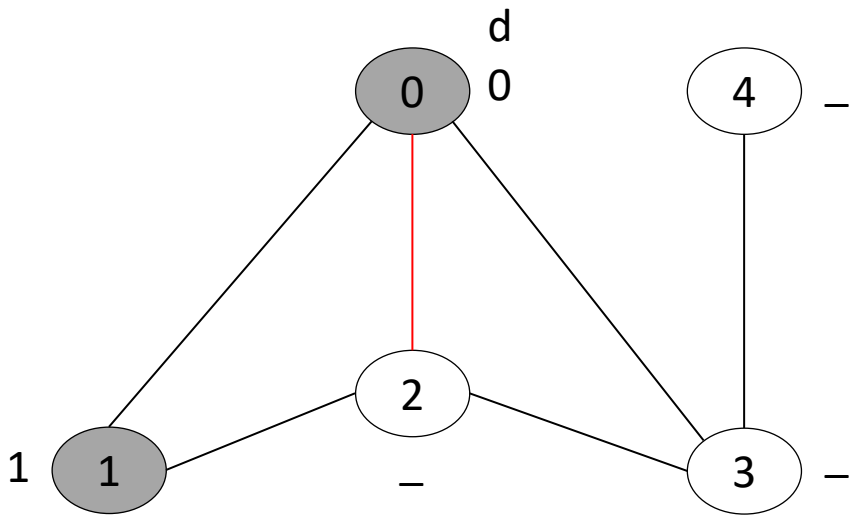
vértice atual → u = 0

Fila: 1 -

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = (0 - 2)

v = 1



Busca em Largura

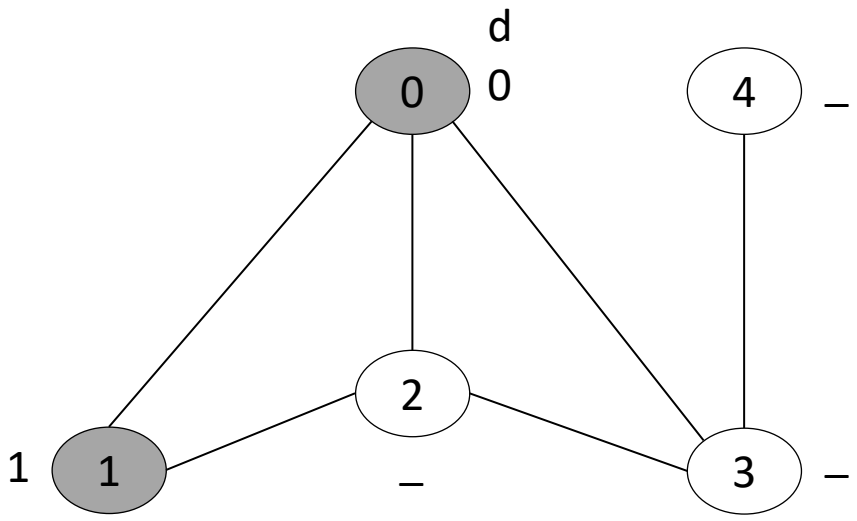
vértice atual → u = 0

Fila: 1 -

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = (0 - 2)

v = 2



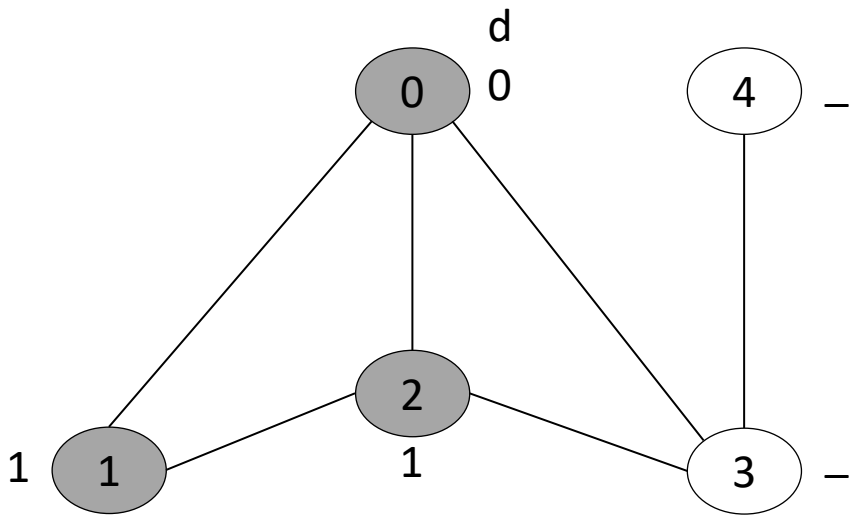
Busca em Largura

vértice atual → u = 0

Fila: 1 - 2 -

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = (0 - 2)
v = 2



Busca em Largura

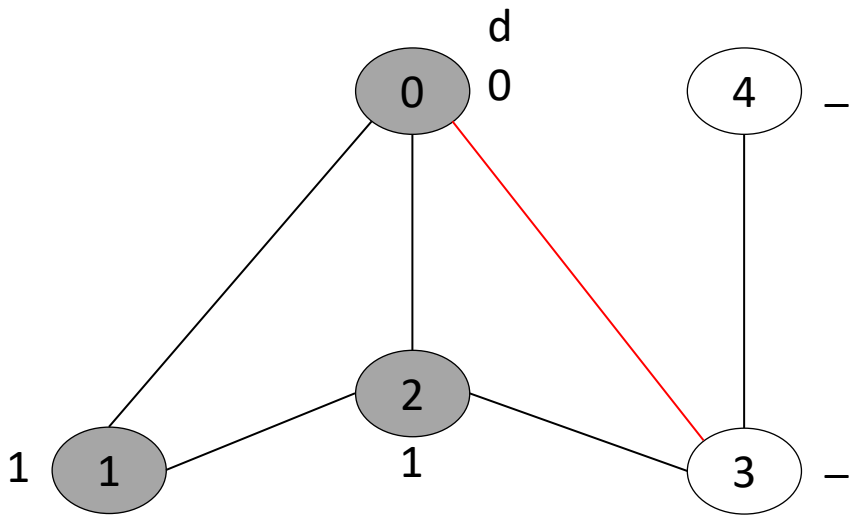
vértice atual → u = 0

Fila: 1 - 2 -

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = (0 - 3)

v = 2



Busca em Largura

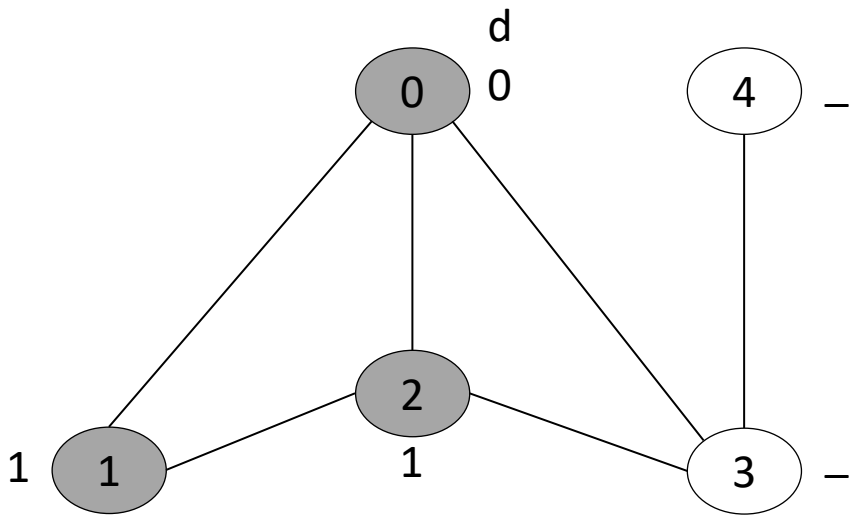
vértice atual → u = 0

Fila: 1 - 2 -

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = (0 - 3)

v = 3



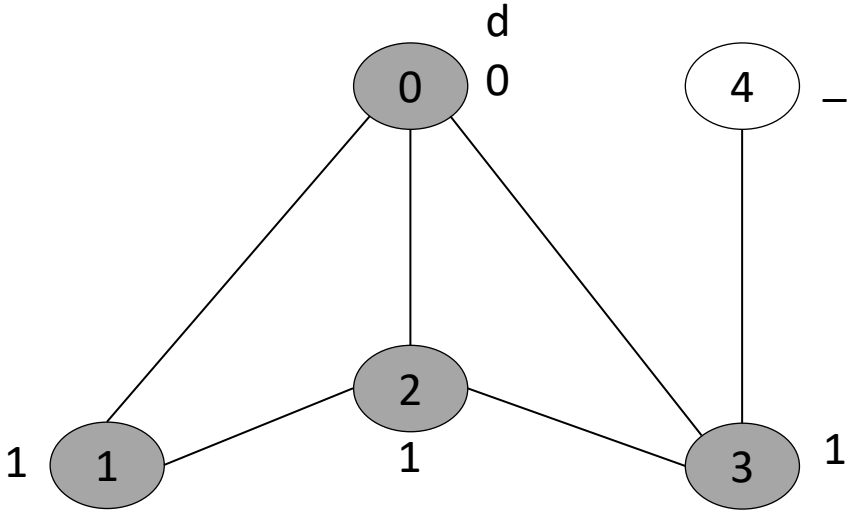
Busca em Largura

vértice atual → u = 0

Fila: 1 - 2 - 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = (0 - 3)
v = 3



Busca em Largura

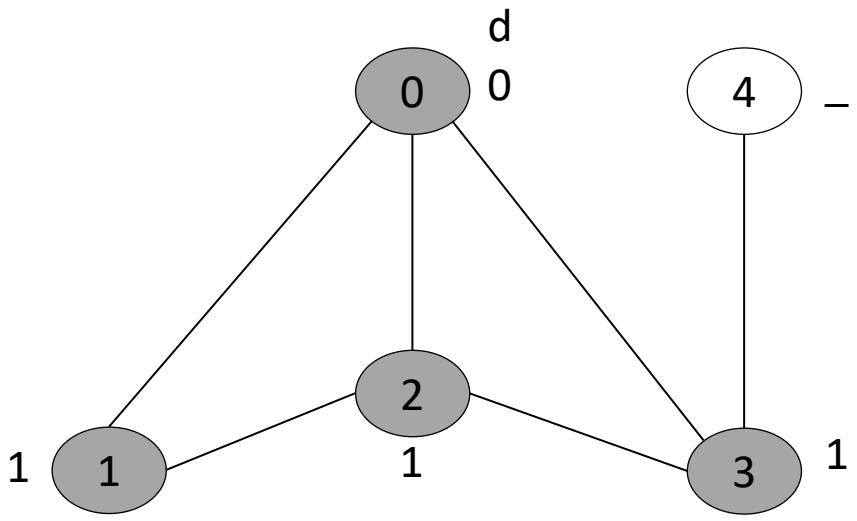
vértice atual → u = 0

Fila: 1 - 2 - 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = NULL

v = 3



Busca em Largura

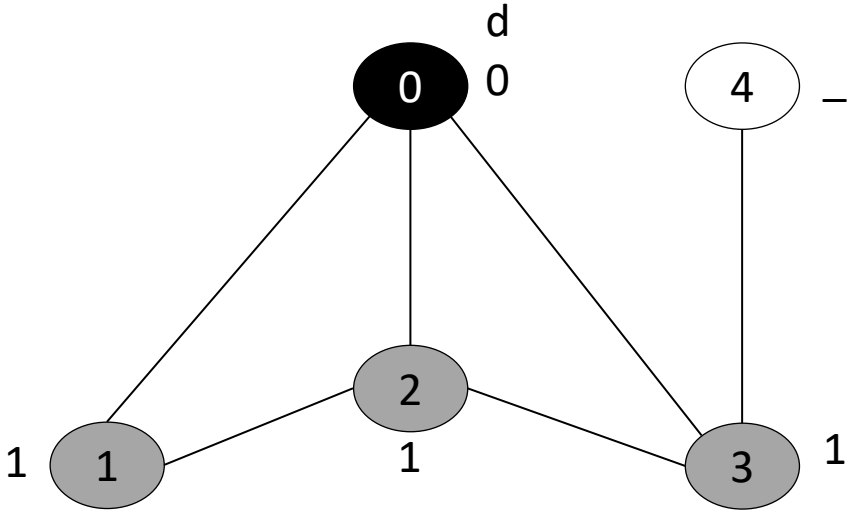
vértice atual → u = 0

Fila: 1 - 2 - 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = NULL

v = 3



Busca em Largura

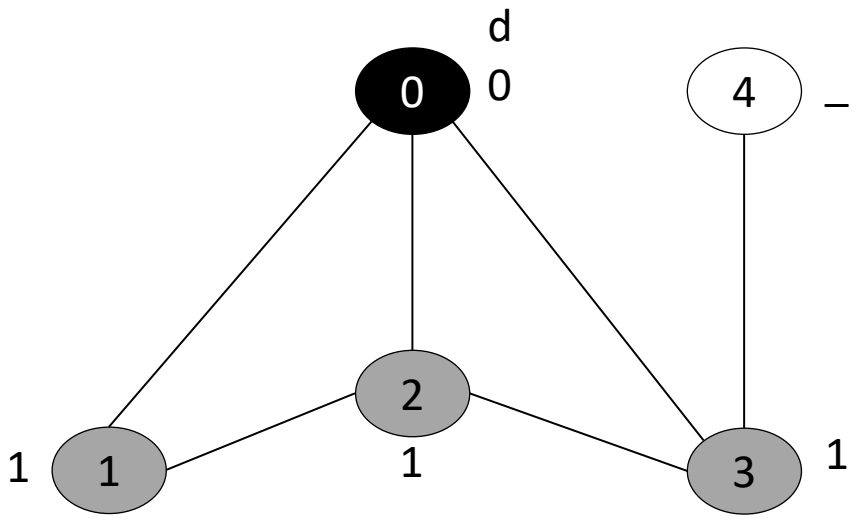
vértice atual → u = 0

Fila: 1 - 2 - 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
          {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = NULL

v = 3



Busca em Largura

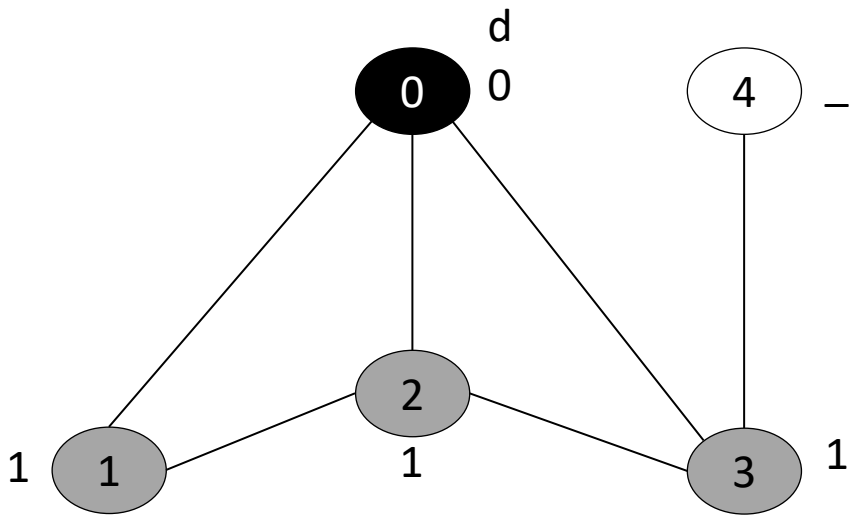
vértice atual → u = 1

Fila: 2 - 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = NULL

v = 3



Busca em Largura

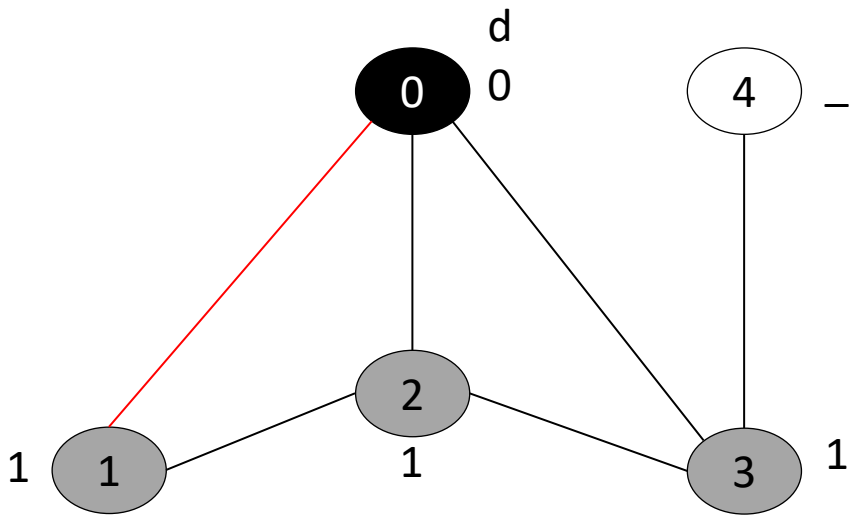
vértice atual → u = 1

Fila: 2 - 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = 1 - 0

v = 0



Busca em Largura

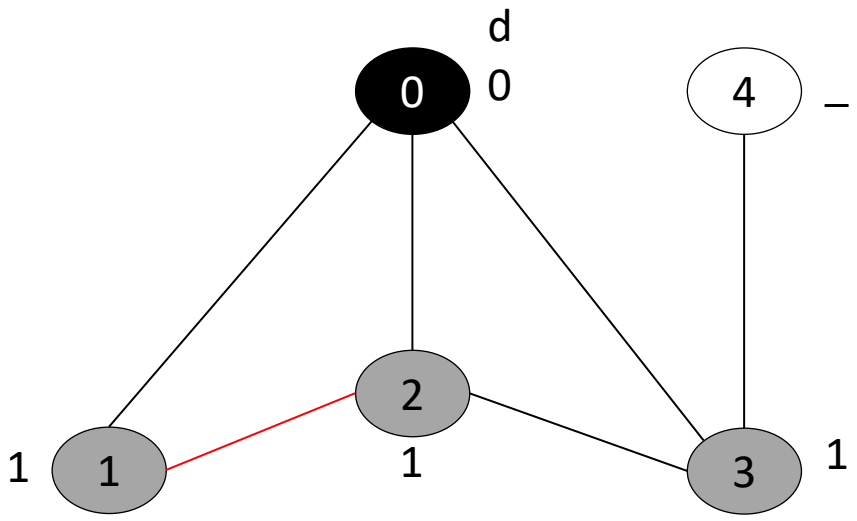
vértice atual → u = 1

Fila: 2 - 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = 1 - 2

v = 0



Busca em Largura

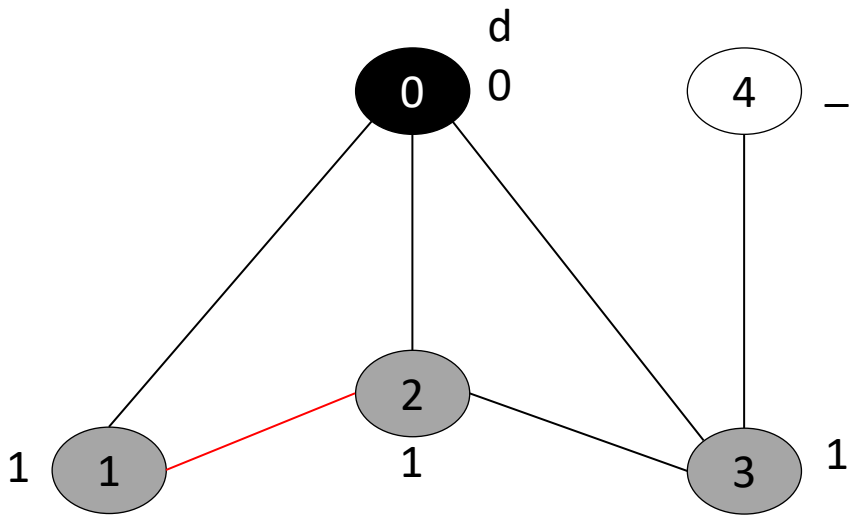
vértice atual → u = 1

Fila: 2 - 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = 1 - 2

v = 2



Busca em Largura

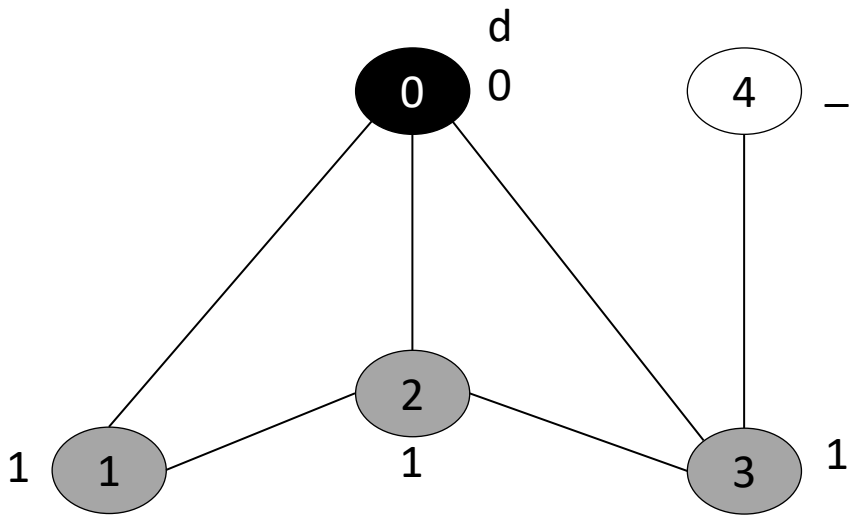
vértice atual → u = 1

Fila: 2 - 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = NULL

v = 2



Busca em Largura

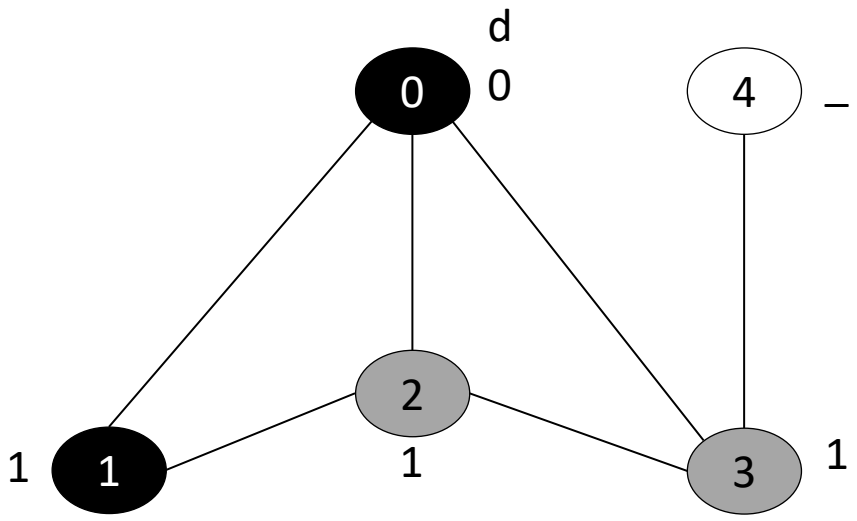
vértice atual → u = 1

Fila: 2 - 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = NULL

v = 2



Busca em Largura

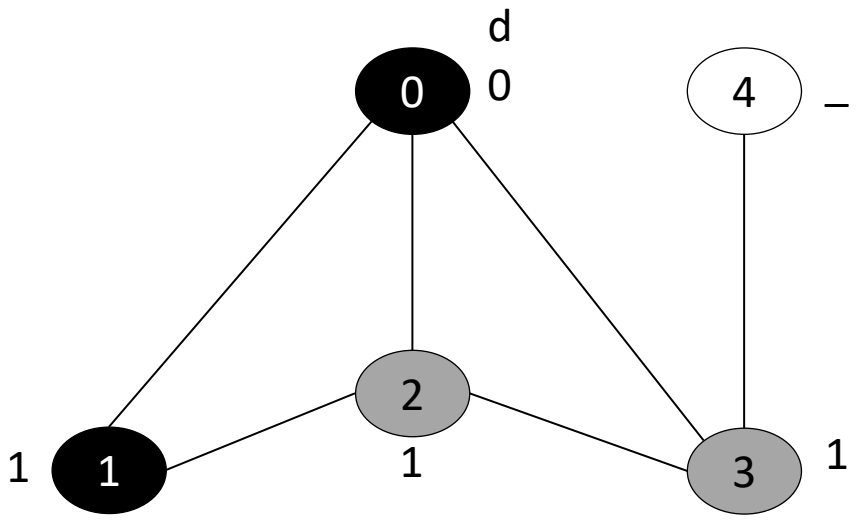
vértice atual → u = 2

Fila: 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
          {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = NULL

v = 2



Busca em Largura

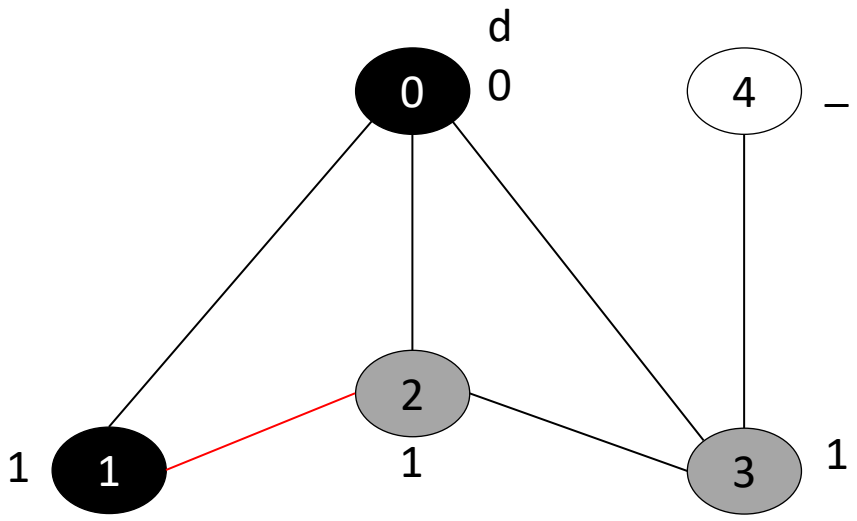
vértice atual → u = 2

Fila: 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = 2 - 1

v = 1



Busca em Largura

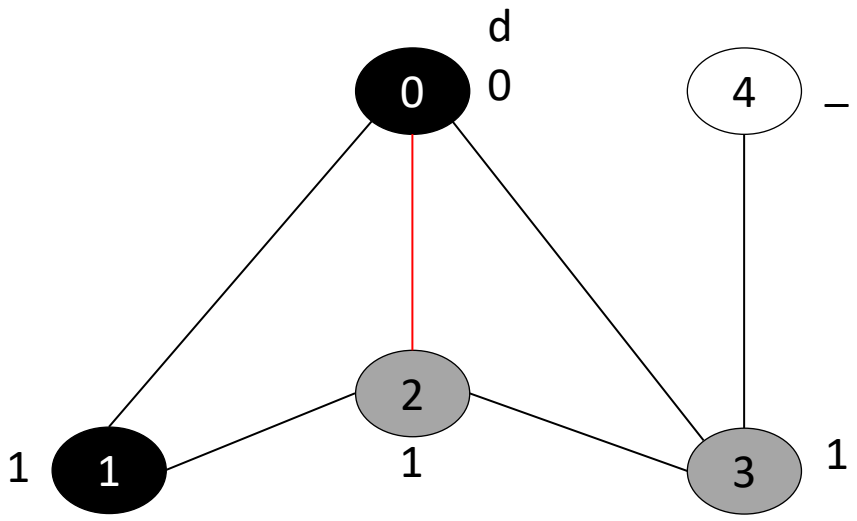
vértice atual → u = 2

Fila: 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = 2 - 0

v = 0



Busca em Largura

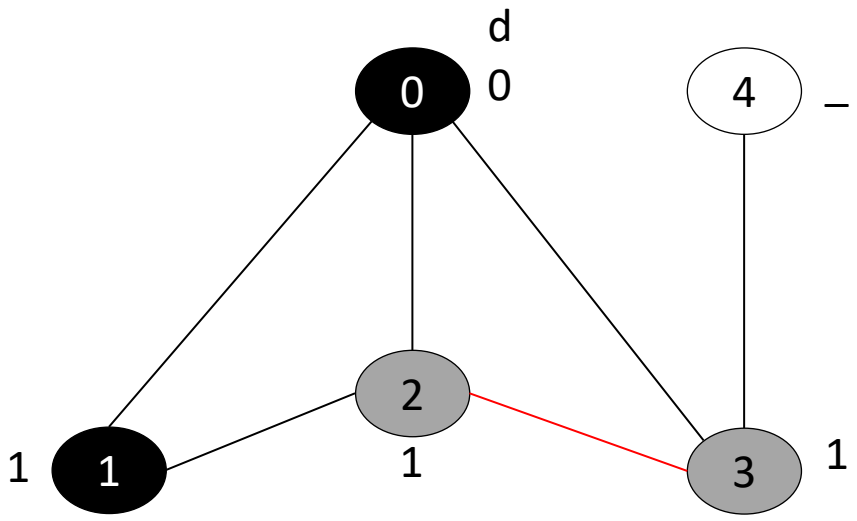
vértice atual → u = 2

Fila: 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = 2 - 3

v = 3



Busca em Largura

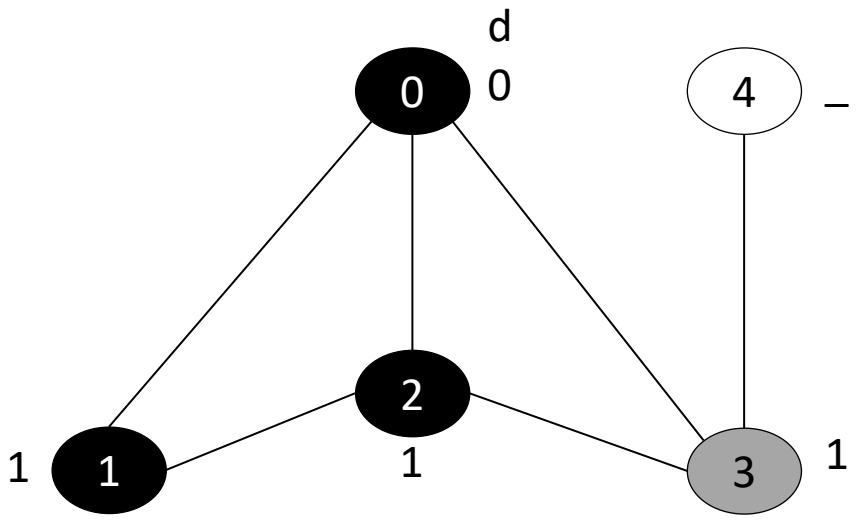
vértice atual → u = 2

Fila: 3

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = NULL

v = 3



Busca em Largura

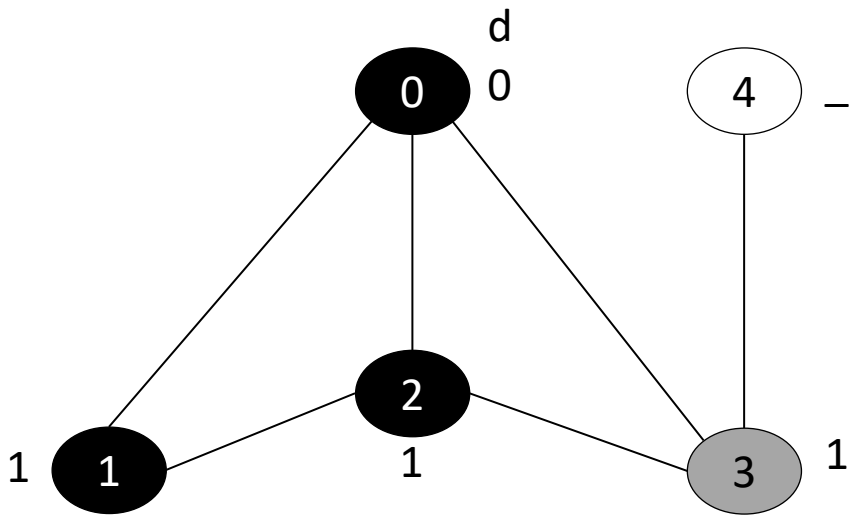
vértice atual → u = 3

Fila:

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
          {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = NULL

v = 3



Busca em Largura

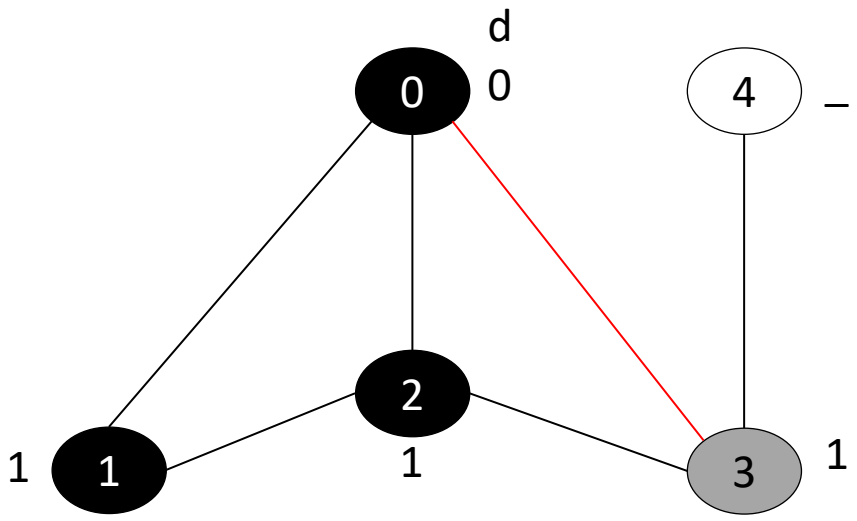
vértice atual → u = 3

Fila:

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = 3 - 0

v = 0



Busca em Largura

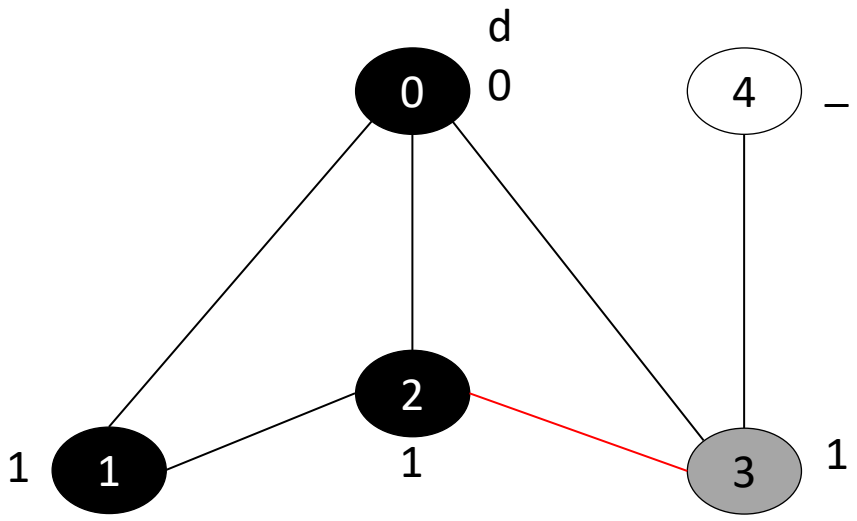
vértice atual → u = 3

Fila:

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = 3 - 2

v = 2



Busca em Largura

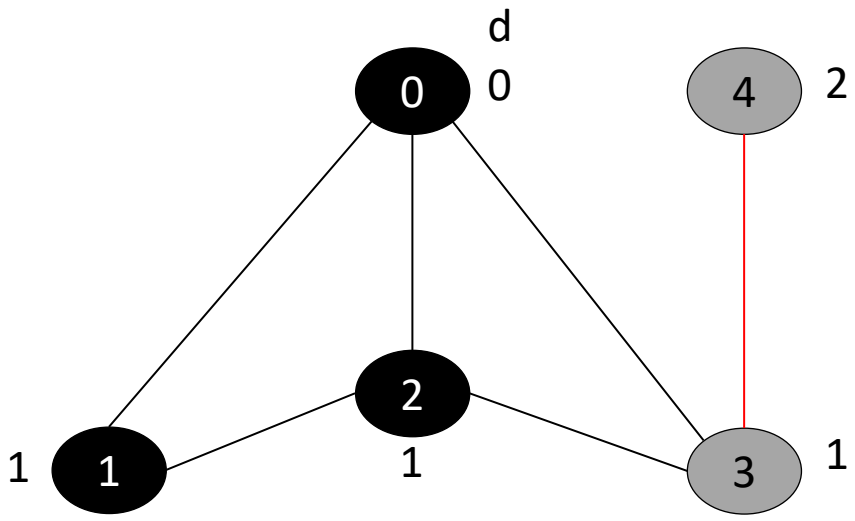
vértice atual → u = 3

Fila: 4

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = 3 - 4

v = 4



Busca em Largura

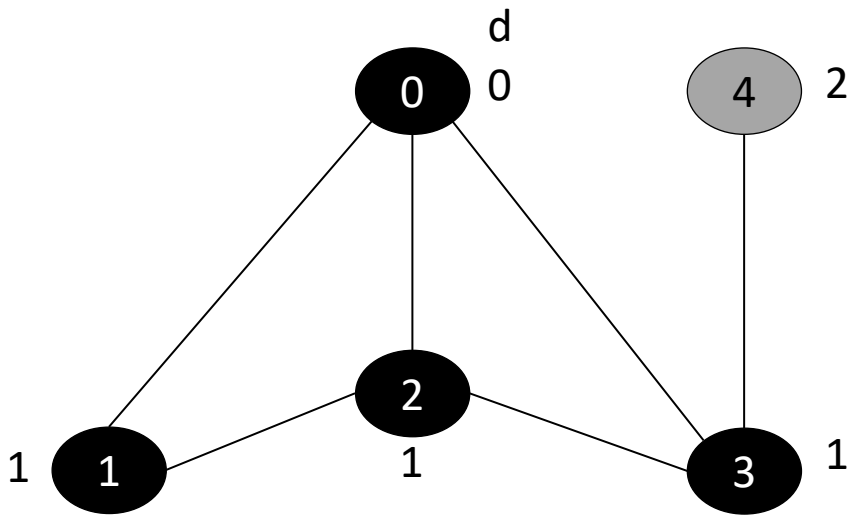
vértice atual → u = 3

Fila: 4

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = NULL

v = 4



Busca em Largura

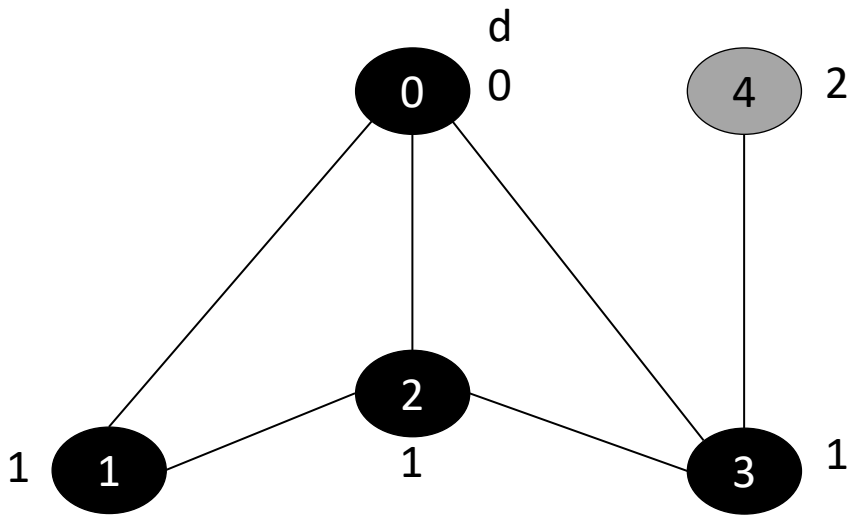
vértice atual → u = 4

Fila:

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
          {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = NULL

v = 4



Busca em Largura

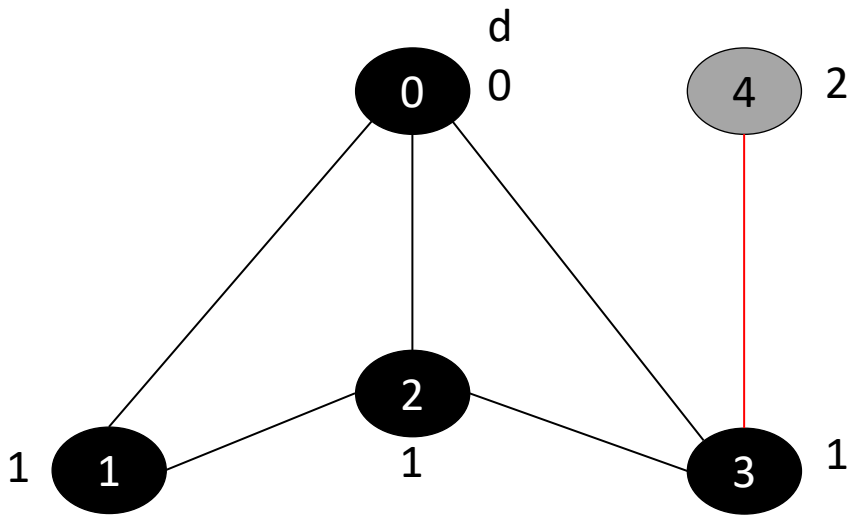
vértice atual → u = 4

Fila:

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = 3 - 4

v = 4



Busca em Largura

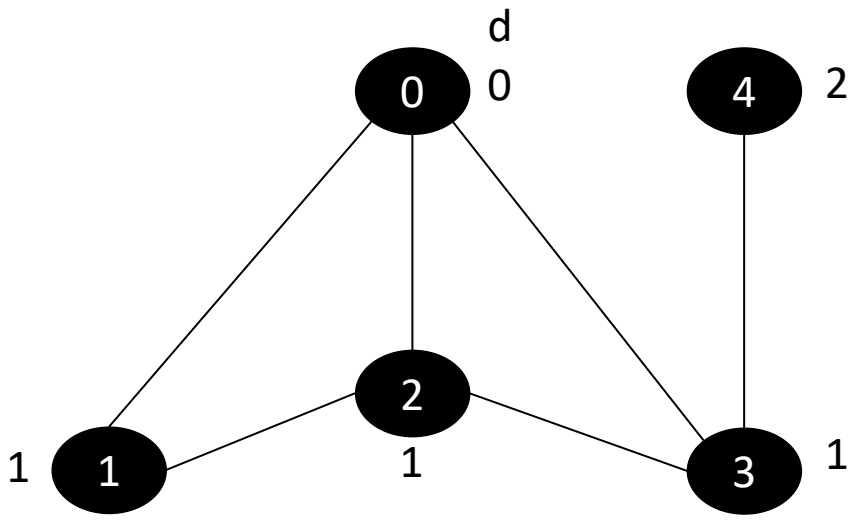
vértice atual → u = 4

Fila:

```
private void visitaBfs (int u, int []cor) {
        cor[u] = cinza; this.d[u] = 0;
        FilaRef fila = new FilaRef (); fila.enfileira (u);
        while (!fila.vazia ())
        {
           int aux = (int)fila.desenfileira (); u = aux;
           if (!this.grafo.listaAdjVazia (u))
           {
              Grafo.Aresta a = this.grafo.primeiroListaAdj (u);
              while (a != null)
              {
                 int v = a.get_v2();
                 if (cor[v] == branco) {
                    cor[v] = cinza; this.d[v] = this.d[u] + 1;
                    this.antecessor[v] = u; fila.enfileira (v);
                 }
                 a = this.grafo.proxAdj (u);
              }
           }
           cor[u] = preto;
        }
}
```

a = NULL

v = 4



Busca em Largura - Análise (para listas de adjacência)

- O custo de inicialização do primeiro anel no método $buscaEmLargura$ é $O(|V|)$.
- O custo do segundo anel é também $O(|V|)$.
- Método $visitaBfs$: enfileirar e $desenfileirar$ têm custo $O(1)$, logo, o custo total com a fila é $O(|V|)$.
- Cada lista de adjacentes é percorrida no máximo uma vez, quando o vértice é $desenfileirado$.
- Desde que a soma de todas as listas de adjacentes é $O(|A|)$, o tempo total gasto com as listas de adjacentes é $O(|A|)$.
- Complexidade total: é $O(|V| + |A|)$.

Caminho mais curto

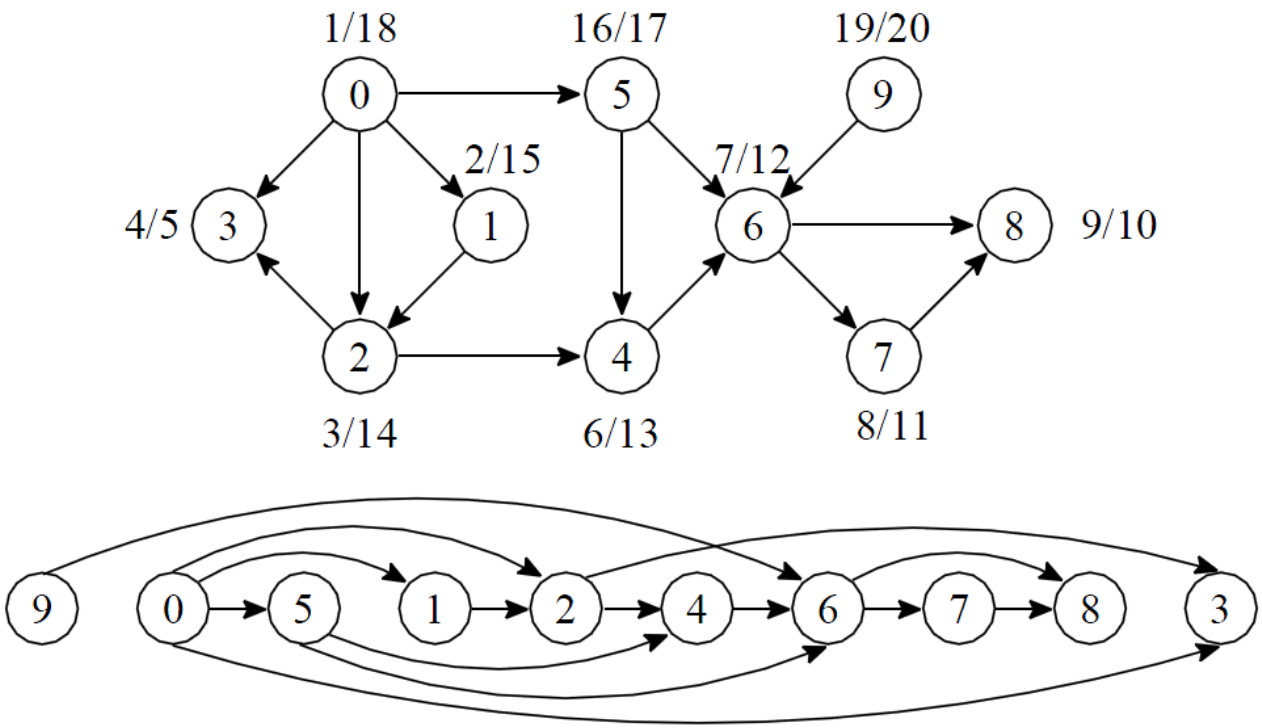
- A busca em largura encontra o caminho mais curto entre dois vértice u e v.
- O caminho entre dois vértices quaisquer fica armazenado no vetor antecessor.
- O programa abaixo imprime os vértices do caminho mais curto entre o vértice origem (do bfs) e outro vértice qualquer do grafo (a distância aqui não considera peso de arestas).

```
public void imprimeCaminho (int origem, int v) {
        if (origem == v)
           Console.WriteLine (origem);
        else if (this.antecessor[v] == -1)
           Console.WriteLine ("Nao existe caminho de " + origem + " ate " + v);
        else {
           imprimeCaminho (origem, this.antecessor[v]);
           Console.WriteLine (v);
        }
```

Ordenação Topológica

- Grafos direcionados acíclicos pode ser usados para indicar precedência de eventos
- Uma aresta direcionada (u, v) indica que a atividade u tem que ocorrer antes da atividade v
- Os vértices ordenados topologicamente aparecem em ordem inversa aos seus tempos de término na busca em profundidade

Ordenação Topológica



[Ziviani, 2006]

Ordenação Topológica

- Algoritmo para ordenar topologicamente um grafo direcionado acíclico $G = (V, A)$.
  - Aplicar a busca em profundidade no grafo G para obter os tempos de término $t[u]$ para cada vértice u.
  - Ao término de cada vértice, insira-o na frente de uma lista linear encadeada.
  - Retornar a lista encadeada de vértices.

- A Custo $O(|V| + |A|)$, uma vez que a busca em profundidade tem complexidade de tempo $O(|V| + |A|)$ e o custo para inserir cada um dos $|V|$ vértices na frente da lista linear encadeada custa $O(1)$.

Ordenação Topológica

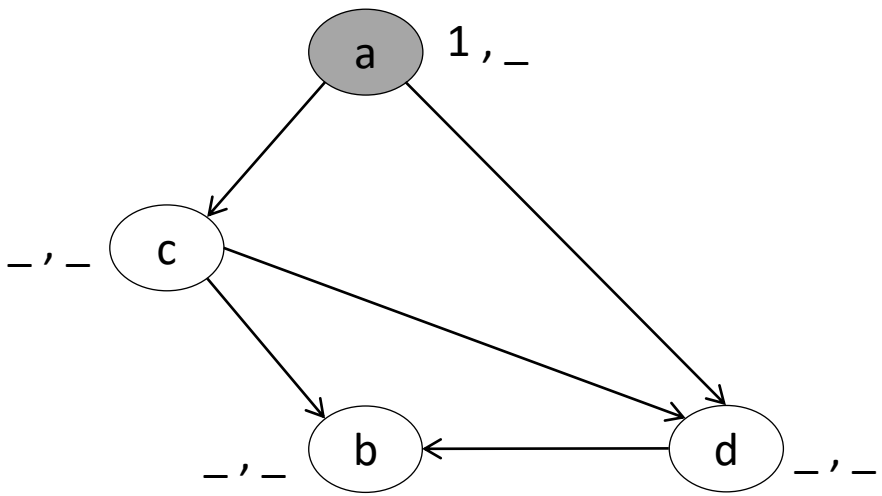
- Deve-se inserir uma chamada ao método $inserePrimeiro$ no método $buscaDfs$, logo após o momento em que o tempo de término $t[u]$ é obtido e o vértice é pintado de preto.
- Ao final, basta retornar a lista obtida.

```
// Insere antes do primeiro item da lista
public void inserePrimeiro (int v) {
        Celula item = new Celula(v, 0);

        CelulaRef aux = this.primeiro.prox;
        this.primeiro.prox = new CelulaRef();
        this.primeiro.prox.item = item;
        this.primeiro.prox.prox = aux;
}
```

Ordenação Topológica

- Exemplo (grafo deve ser orientado e acíclico):

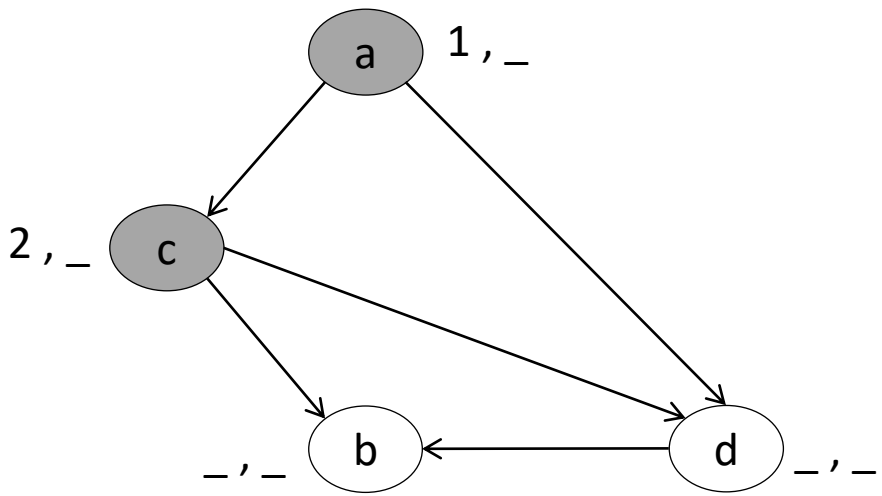
Lista: null



Ordenação Topológica

- Exemplo (grafo deve ser orientado e acíclico):

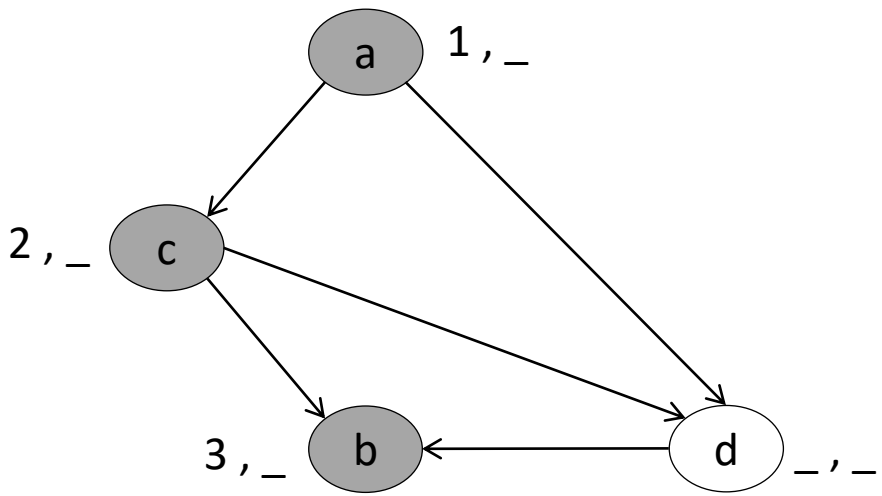
Lista: null



Ordenação Topológica

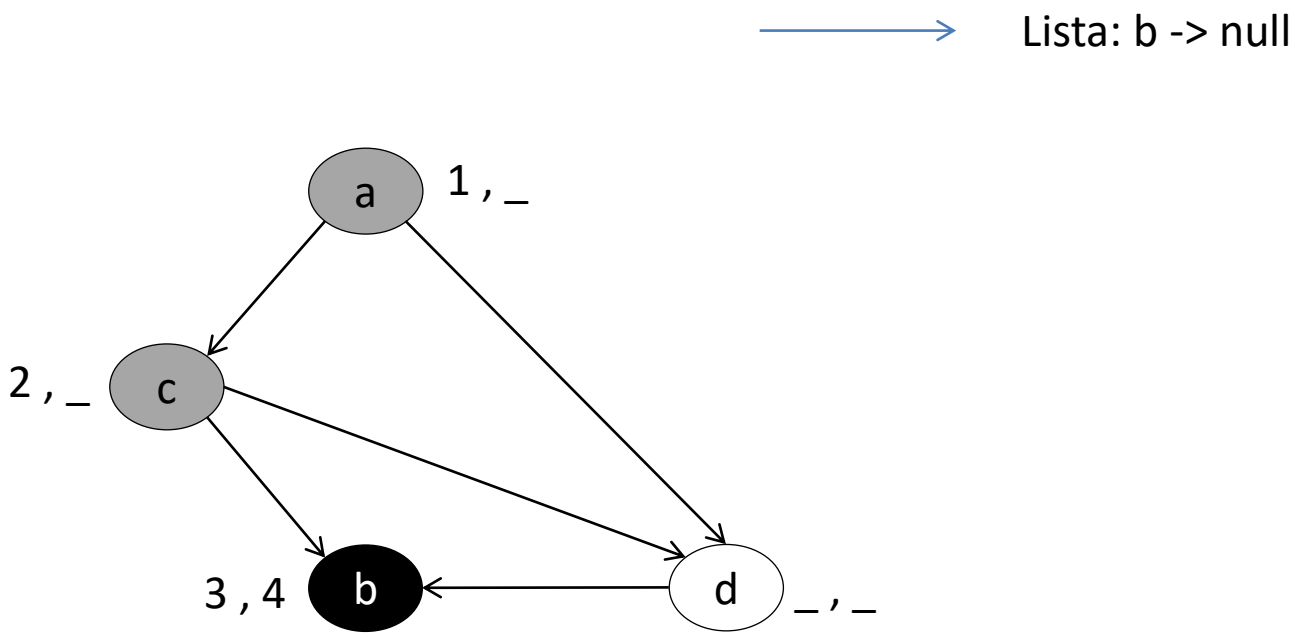
- Exemplo (grafo deve ser orientado e acíclico):

Lista: null



Ordenação Topológica

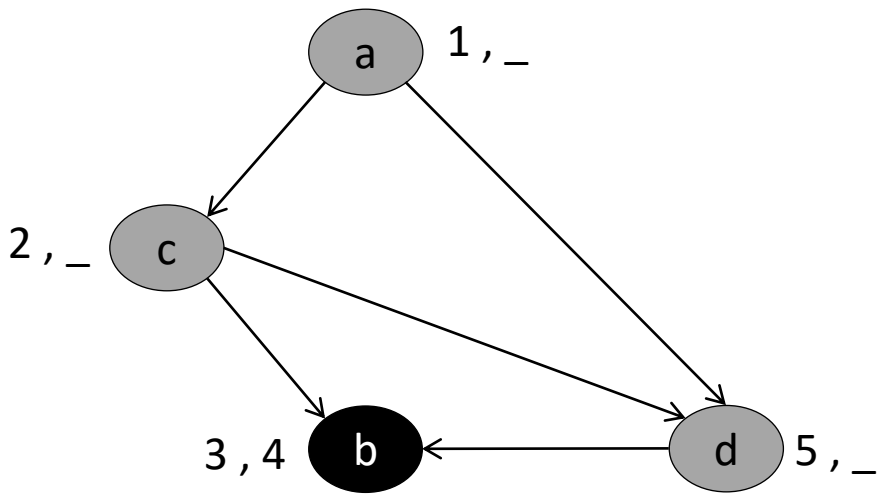
- Exemplo (grafo deve ser orientado e acíclico):



Ordenação Topológica

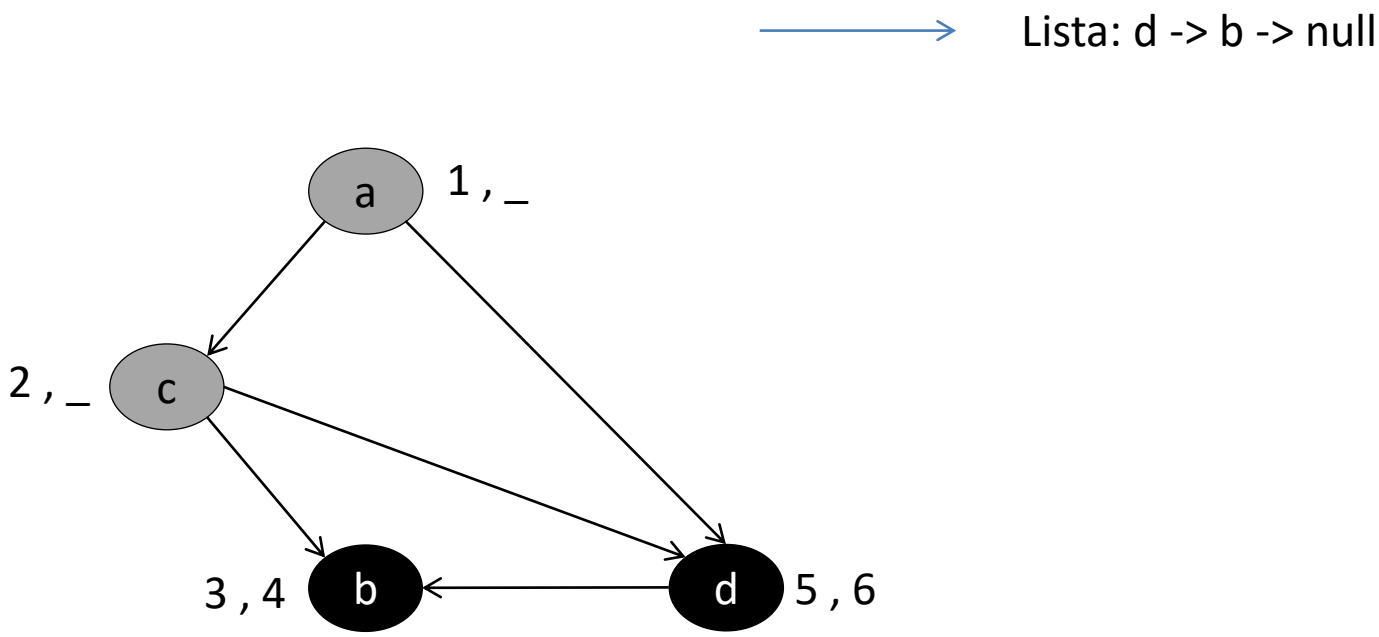
- Exemplo (grafo deve ser orientado e acíclico):

Lista: b -> null



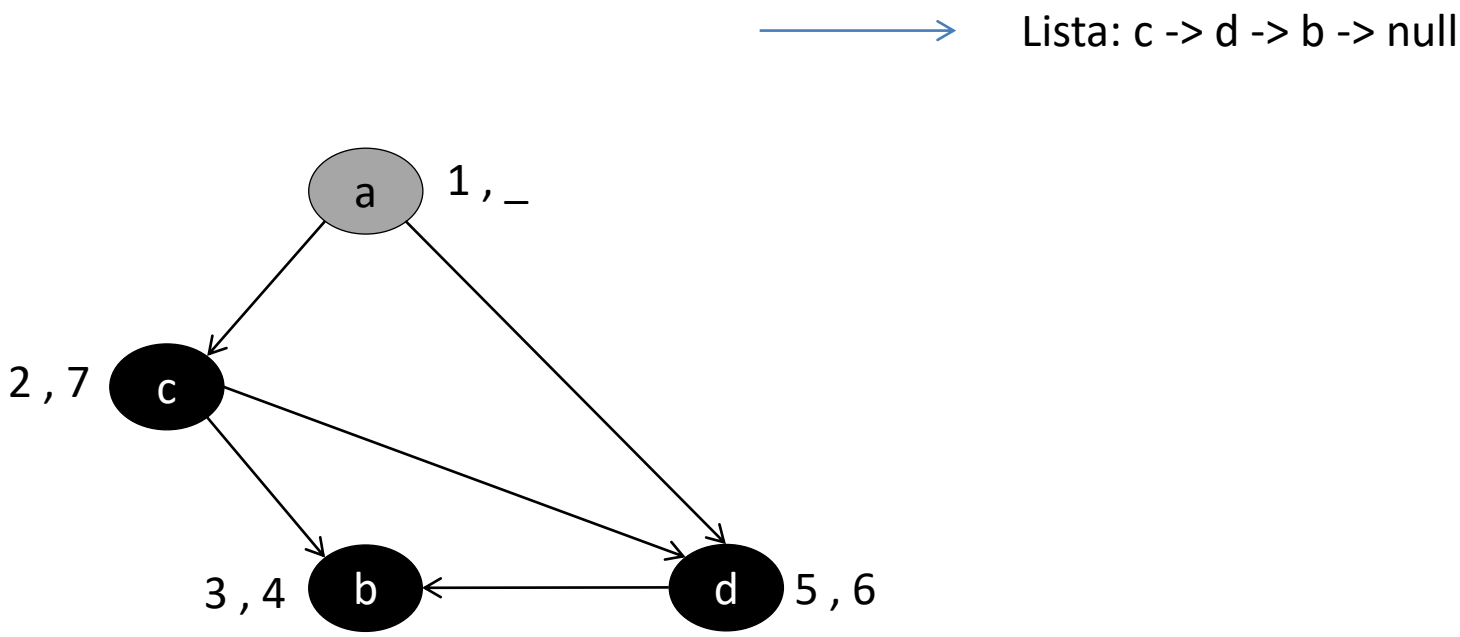
Ordenação Topológica

- Exemplo (grafo deve ser orientado e acíclico):



Ordenação Topológica

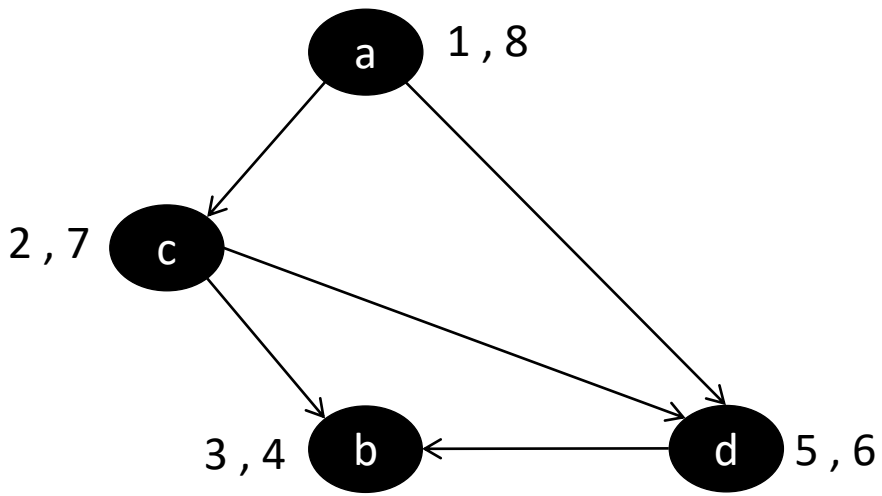
- Exemplo (grafo deve ser orientado e acíclico):



Ordenação Topológica

- Exemplo (grafo deve ser orientado e acíclico):

Lista: a -> c -> d -> b -> null



Ordenação Topológica

- Implementem a solução de ordenação topológica dada em sala. Lembre-se que a ordenação topológica é aplicada apenas em grafos orientados e acíclicos.
- Utilize o algoritmo de detecção de ciclos em grafos para verificar se o grafo é acíclico.
- Altere a implementação de busca em profundidade para fazer a ordenação topológica.

Bibliografia

[Cormen et al., 2009] Introduction to Algorithms, 3rd edition. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. 2009, 1312 pp. MIT Press, ISBN 9780262033848.

Cormen et al., 2002] Algoritmos: Teoria e Prática – Tradução da segunda edição em inglês. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. 2002, 936 pp. Editora Campus, ISBN 8535209263.

BOAVENTURA NETTO, Paulo Oswaldo; JURKIEWICZ, Samuel.
Grafos: introdução e prática. Editora Blucher. 2009. ISBN: 9788521204732.

[Sedgewick, 1992] Algorithms in C++. Robert Segdewick. c1992, 656 pp. Boston: Addison Wesley, ISBN 0201510596

Marco Goldbarg; Elizabeth Goldbarg. Grafos - conceitos , algoritmos e aplicações . Campus. 2012.

[Ziviani, 2006] Projeto de Algoritmos com Implementações em Java e C++.
Nivio Ziviani. 2006, 642 pp. Editora Thomson, ISBN 8522105251.

Robin J. Wilson. Introduction to graph theory. Longman, 1996